Agent-native 开发 [08-Raw/26生成式软件工程/NJU]

> 原视频：https://www.bilibili.com/video/BV11LWw6VEHW · 课程合集：https://space.bilibili.com/202224425/lists/8942017 · 讲者：蒋炎岩（NJU《生成式软件工程》2026）

全课主线

这节课的核心主张是：Agent-native 开发不能理解为“把任务丢给模型一次做完”，而要把软件开发看成在 intent→spec→implementation 的设计/搜索空间中做状态迁移，用 agent loop、harness、BFS/DFS、并行和 CI/CD 门控来管理 agent。讲者想让听众相信，模型能长程执行但会漂移、扶 slop，所以必须把“开发者不可信”落实成可自动检查的硬门控与文档/代码一致性机制。进一步，人若没有反复试错、好项目和 token 机会，就会被 agent 的性价比逻辑“斩杀”，因此要去开源中学习真正优秀的 agent-native 项目。

1. 课程过半转入 Agent native 开发，第二个实验要把“算力换智力”往前推；理由是 agent 已能完成长程任务，但解决问题直觉稍差。因此，接下来不是重复个人 agent 产品，而是研究如何用更多算力/搜索补短板。（第 1 部分）
2. 讲者用 PhD 学生和模型 RL 作类比：模型像巴甫洛夫的狗，完成任务才给奖励，容易偏向较短的广度搜索；但 design space 浅层分叉多，不同早期选择会放大到最终结果，必须平衡 BFS 与 DFS。（第 2、5 部分）
3. 软件工程本身也处理同一类空间问题：intent→spec→implementation 都是一对多，每个箭头要靠 review、讨论、迭代收敛；而 AI 的最大能力是从任意阶段开始向下推进，并生成版本比较趋势。（第 2、3 部分）
4. 需求能拆成两个 subsystem 时，就可以启动并行 agent；AI 成本足够低时，可先让它搓原型，再用典型 use case 评审。但这要求有 agent loop，否则无法支撑原型构建。（第 3、4、5 部分）

5. 一个 ReAct 型 agent 的主循环可以很简单，约 150 行，带 bash tool 就能跑；但模型会自己产生行动、反问你，它的自选行为不一定是你要的，因此需要 harness 约束可做范围。今天 harness 的版本答案被讲者称为 Ralph loop：同一 prompt 反复检查、修改，直到无法改进。（第 6、7、8 部分）

6. （整理者串联）Ralph loop 把“一把头做完”拆成很多小步后，workspace 就可看成一个非确定状态机：同一状态可换 GPT、Claude、DeepSeek 或不同操作，于是回到搜索过程。再由此推出，每次开发都是状态迁移，同一状态可并行分叉、最后 git merge，所以 agent 能并行，long-horizon task 也能越做越好。（第 8、9、10 部分）

7. 但现实模型容易漂移、被已有代码污染，又有遵循指令/舔狗式倾向，会倾向于扶起 slop 而不是重构；长任务遇到 compact 还会降智。因此讲者的办法是让模型停留在 design space，用 prompt 或 loop 做 BFS/DFS 管理，而 ReAct loop 只是线性管理。（第 8 部分）

8. 为了防漂移，loop 里要每轮先检查当前状态，只推进一个明确步骤，然后停下验证、评估、review；这来自软件工程里“开发者不可信”的 first principle。由此自然导出 CI/CD：每个状态推进后都检查，不通过就拒绝推进，全部通过才发布。（第 12、13、14 部分）

9. 在 Agentic AI 时代，CI 含金量更高，因为 agent 是“最不被信任的开发者”；不能只靠 AGENTS.md 提醒，因为上下文膨胀会让 agent 注意力涣散、滑坡。所以要把 lint、测试、文档长度、术语、接口/文档一致性等可判定规则变成 pre-commit、CI 等硬门控。（第 15、16、17、21、22、23、24 部分）

10. 文档与代码漂移是滑向 slop 的核心风险：agent 把思考流程写进 doc.md，换 agent 或 compact 后无人记得文档与 implementation.py 要保持一致。DeepSeek Harness 用配对契约、i18n.yaml、hash/git 历史、AI patch 翻译和术语门控保证中英文及文档/代码一致性，并用关系组织项目、IDE 投影、渐进披露来维持可维护体验。（第 16、20、21、22 部分）

11. Codex/Anthropic 的经验显示，公开 CI 往往只是基础 lint/测试/发布门控；真正支撑 AI-native 项目的是背后庞大的 agent-native 门控、orchestration、agent review、log book/dashboard 等机制。讲者自己的 architecture.md、log book、dashboard、master agent 设想也在同一方向上。（第 17、18、19、23 部分）

12. 这些 agent-native 经验来自真实实践、失败和开源，不是课堂推演；如果没有 token 和反复试错机会，学生就更难追上 agent，旧培养体系会按性价比被“斩杀”。因此讲者最后让大家去互联网找真正优秀的项目，并明确从 DPC Harness / DeepSeek Harness 这类开源项目看起。（第 22、24、25、26、27 部分）

讲者的结论/号召是：把开发当成受门控管理的状态迁移，亲自学、亲自搭 agent-native loop，并去开源社区找真正优秀的项目，从 DPC Harness / DeepSeek Harness 开始。

第 1 部分



课程过半，今天转向 Agent native 开发

- 课程已进行到差不多一半，讲者今天进入 **Agent native 开发**（原识别：A阵 native 开发）怎么做。
- 今天早上 **TEO（原识别，未确定）** 宣布今天要 **reset**。

reset 前挂长任务：goal、复杂数学证明与 token 额度

- 讲者问听众：有没有设置一个 **goal**（原识别：go），让它去跑很长的任务?
- 他自己的例子：积攒了一些非常消耗 token 的任务，例如一个复杂的数学证明；他自己也不知道怎么证，但可以让 agent 去证。
- 代价：这种任务可能把一周额度全部跑完。
- 操作：离开办公室前直接上一个 goal，让 agent 继续证明；等今天 reset（以及第二个实验发布）后再继续/释放额度。
- 讲者把“今天 reset”和“第二个实验发布”放在一起说。

第二个实验：从个人 Agent 产品推进到“算力换智力”

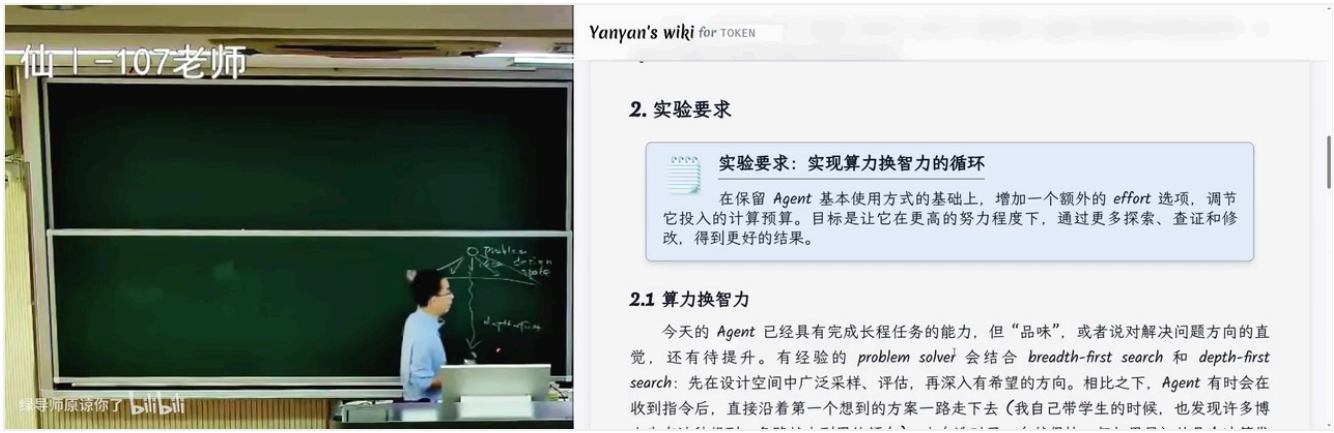
- 第二个实验和第一个实验有点不一样。
- 第一个实验：有点像产品，做个人 agent。做个人 agent 的过程中，会发现一些产品发布，例如 **Grok bot**（原识别：Grog bot / grb board / grp boss，疑似同一产品）；它开课前已发布，最近才火。
- 讲者把日常 job 迁移到自己的 Grok bot：
  - 每天给小朋友生成 **podcast**，开车时听；背后调用 **Claude Code** 的 **Opus 4.5** 模型（原识别：cloud code 的 opus s5.5）。
  - 每天收到 **morning paper** 推送，例如 **CVPR** 论文；这个 **prompt**（原识别：PROMPTY）是讲者调过的，所以直接让 AI 总结论文会得到不太一样的结果。
  - 还有 **GitHub trending**（原识别：github training）等。
- 第二个实验会取第一个实验里的一小部分：第一个实验中的 agent 可以帮你 **digest 邮件**，或根据你的数据库回答问题。
- 实验2要把 **“算力换智力”** 稍微向前推进一点。
- 讲者理由：Agent 虽然具备 **长程任务** 完成能力，但解决问题的 **直觉** 稍差一点。
- （整理者串联）所以实验2不是单纯重复个人 agent 产品，而是用更多算力/搜索去补这种直觉短板。

类比 PhD 学生：能 DFS 往下做，但要在 design space 里找解

- 讲者认为 Agent 有时很像 **PhD 学生**（原识别：pg student / p h d student）。
- 场景：有一个有趣问题，可能是研究问题；学生想到一个方法，这个方法肯定有合理性。
- 有了 problem 和 idea，就可以不停往下做：用 **DFS（深度优先搜索）**（原识别：DEFAST）的方法，先把 idea 实现成原型，再优化，再改进实验结果，继续。
- 但讲者强调这里有一个 **design space**。
- 他常对 PhD 学生说：通过做一个 **deep/narrow** 的具体研究——例如软件工程里 Python 代码某种类型补全问题，一个非常窄且很深的问题——你可以把它做好；但在过程中你会体会到，你其实是在一个很大的 design space 里找到这个问题的一个很好的解。
- 讲者结论：所以我们训练人类、训练模型，两者的东西是稍微有点不一样的。
- （整理者串联）链：Agent 会长程执行、沿一个合理 idea 做 DFS；但直觉弱，可能缺少在大 design space 中比较与选择的能力；PhD 训练则试图让人通过具体 narrow 问题体会 design space。

小结： Agent 能长程执行但直觉弱；“算力换智力”是第二个实验推进方向，博士生类比用来解释人类与模型训练的差异。

第 2 部分







人类训练与模型 RL 不同：模型像巴甫洛夫的狗，完成任务才给奖励

- 讲者从“能不能找到这个问题的好解”切入，指出：人类训练出来的东西和模型训练出来的东西稍有不同。
- 模型强化学习的典型说法是：最后完成任务 → 就给奖励。讲者打比方：像巴甫洛夫的狗一样。
- **（整理者串联）** 这一对比为后文铺垫：模型 RL 很依赖最终任务奖励，而 design space 里的早期分叉会显著影响最终结果，所以不能只靠最后奖励来指导搜索。

Design space 浅层分叉很多，搜索要综合宽度与深度

- design space 里有很多分叉，而且它们都在搜索空间的浅层。
- 不同方案会对最终结果产生很大影响。
- 讲者称这是一个“分叉/分支（转写为 breakfast，疑为 branching）的过程，宽度有限”。
- 搜索必须综合宽度和深度，因为算力有限。
- 逻辑链：
    1. 浅层有多个分叉。
    2. 不同浅层选择会放大到最终结果。
    3. 只走窄或只铺宽都不够。
    4. 算力有限 → 必须做宽度与深度的折中。

Agent loop 的参考：多模型互评比单模型一条路走到底更有效

- 你们会自己做一个 **agent loop**，这个 loop 里也有参考做法。
- 例子：**Humanize**（原转写 HUMHUMANIZE/Humanize，疑似团队/项目名）。讲者说他“一直潜伏在他们的群里”，他们烧了很多 token，研究如何让 **multi-agent 协同**完成更困难的事情。
- 他们最后发现最有效的方式：让 **GPT 和 Claude 互相 review、互相挑刺**。
- 类比人的多样性：要找两个人，而且互补。讲者提到“很容易达成共识”的问题：如果我说这个方案这样会更好，另一个人可能很容易就同意。
- 在**宽度优先搜索**过程中，用两个不同模型：两个不同模型带有不同偏见；讲者说每个人也都是带偏见的。一个模型说往这走，另一个就可能岔开。
- 讲者提到“几轮以后可能找到一个更好的 **solution**”。**（整理者串联）** 这应指两个不同模型互相 review、在宽度上岔开后的搜索过程；若只让一个模型“一路走到黑”，就可能沿单一偏见走，效果不同。
- 技术要点：还要构建**知识库**。

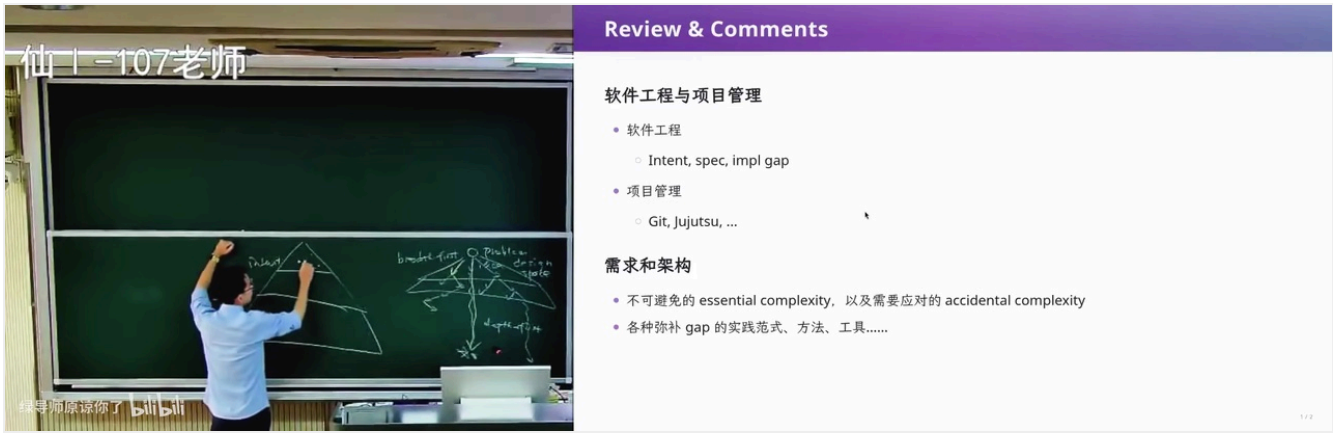
Deeper research 需要更多知识来源，不只是实验 database

- 第一个实验里，你们已经构建了自己的 **database**。
- 如果想做 **deeper research**，需要更多 **knowledge**：实验作业、奖学金申请、个人简历、研究论文、博客、GitHub 项目、从书里摘选的片段等，并放进 **report**。
- 因此这个实验会稍微开放，但基本上仍有框架。
- 讲者说，实验可以选择“摆烂”——直接不做，可能也能 OK，课也可能就这么过去。
- 但如果做过，有 first time 经验，真正观察过 **agent trajectory**，会发现一些有趣的、只有自己才有的体会。

软件工程也解决同一空间问题：intent → spec → implementation 是一对多

- 讲者特意画了图。软件工程解决的问题也是这个空间里的问题：从 **intent → spec（原转写 spark）→ implementation**。
- 他举例：我可以画圈，也可以画三角——说明选择不是唯一的。
- 人的脑容量有限。组一个小团队做一件事，或自己做一个大作业，在 **intent space** 里都只是采样一个点。
- 但实际上 intent space 里有很多点。
- **intent → spec 是一对多；spec → implementation 又是一对多。**
- 这就意味着，如果你有一个 **review**，比如一个 review……

第 3 部分



intent→spec、spec→implementation 都是一对多：每个箭头靠 review/讨论/迭代收敛

讲者先给出结构：

- **intent → spec** 是一对多。
- **spec → implementation** 也是一对多。
- 由此，一个 **review** 可以和参与者来回讨论；经过几轮迭代达成共识后，落到一个点；再到下一个点，又经过几轮迭代，再达成共识。
- **（整理者串联）** 这套结构的关键不是一次确定，而是在每个一对多分叉处局部收敛；收敛结果再进入下一步。

需求阶段：访谈和 checklist 对齐甲方；甲方越说得清，intent 越清楚

- 讲者举例：需求与架构。
- 在需求阶段，可能做 **访谈**、**checklist**，与甲方对齐。
- 如果甲方是说得清楚、对自己想做的事非常清楚的人，就可以走向一个更清楚的 **intent**：到底要做什么。

specification 阶段：与有经验架构师讨论技术选择，多想法可能收敛出很不一样的实现

- 在 **specification** 阶段，可以和**有经验的架构师**讨论：用什么数据库；用不用 **event sourcing**。
- 讲者提到最近某“pad”（原文，术语不确定）开发者站出来说，他们不要用 event sourcing，而用了另一种 **functional design**，也是类似的，需要 **persist everything**。
- 讲者认为，这个过程会产生很多不同想法，最终收敛到一些很不一样的实现上；“一条线走到黑”的实现，在**可维护性**、**可观测性**上会差**十万八千里**。

软件工程解决的就是一种智慧：用 UML 等约束设计空间，让早期达成一致；工具链支撑日常实践

- 讲者说，软件工程解决的就是一种智慧。比如用 **UML** 建模；当你用 UML 建模时，就是在约束这个空间的行为，让你在项目早期就可以和别人达成一致。
- 也有很多工具：从 **IDE / VS Code IDE**，到管理版本状态的 **git worktree**，还有更先进的工具和无数工具，讲者说举不过来。
- 如果你或 agent 每天在 **GitHub** 上给你推一个 **GitHub trending project**，进去看，会发现每个项目用了不同**技术栈**，里面有很多好东西；每天花几分钟，比如追问一下，就能提升自己的技术积累和品味。讲者说这是前一半讲过的内容。

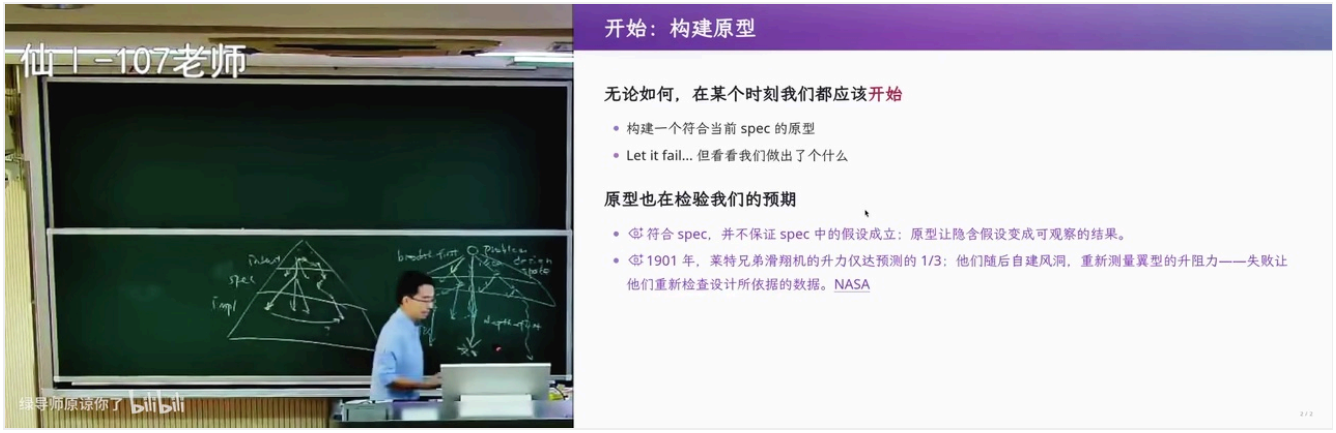
AI 的最大能力：从任意阶段随时开始，让 AI 向下推进并生成版本比较趋势

- 讲者认为 AI 给我们的最大能力，还是**随时随地都可以开始**。
- 你可以想着可以学、可以想，但不需要想太多；在任何一个阶段，哪怕最开始只有一个想法，就可以直接让 AI 向下做。
- AI 可能到不了那么深：做到最后可能 stop、维护不下去，或者你觉得看不下去了。
- 在讨论过程中，也可以随时让 AI 生成一个版本；这个版本可以比较：是变好了还是变坏了，趋势上是变好还是变坏。若变坏了，就再讨论，再让它做一个版本。

AI 可承担员工甚至团队职责；需求分解好时可启动并行 agent

- 讲者说，AI 实际上承担了一个员工，甚至不只是一个员工，甚至一个团队的职责。
- 如果需求可以分解成两个 **subsystem**、需求模块分得比较好，就完全可以启动两个**并行 agent**。
- 讲者说，像他现在实际开发一个项目的时候……

第 4 部分





需求能拆成两个 subsystem，就可启动并行 agent

讲者的推理链：

1. 需求可以分解为两个 **subsystem**；
2. 需求模块分得比较好；
3. → 完全可以启动两个并行 **agent**。

讲者自己的做法：

- 实际开发一个项目时，他有一个 **tmux 窗口**（原转写“TMARS 窗口”），分四份，就有四个 agent。
- 他最多只开四个并行：不是技术上限，而是个人上限——不然大脑跟不上 AI 输出速度。
- 他负责**监工**，因此可以很快在**搜索树**（原转写“搜索书”）上探索，也可以推进并行的需求想法。

AI 成本足够低时，先让它搓原型，再用典型 use case 评审

- 当前阶段可能只有一个 **intent** 或 **spec**；如果 AI 成本足够低，就可以先让它承担，帮助预测。
- 他举的预测问题：这个架构如果“这个人来了”会怎样？用户多了以后会怎样？两个模块之间有没有之前没想到的交互？
- 讲者的判断：很多问题只有真正实现出来以后，才意识到“这个地方设计出了一个小问题”。
- 因此他的做法是 **let it fail**（原转写“let it feel”）：我现在没想清楚不要紧，你给我搓一个；搓完以后，把 **use case** 构建出来，再评审一下。
- 讲者认为：它可以替代人类的智力；有些事情他做的就可以让 AI 做了。
- 所以 AI 给了**构建原型**的非常好的能力；而要用 AI 构建非常好的原型，就需要 **agent loop**。

早期模型有幻觉，所以 Devin 式自主 agent 当时被观望

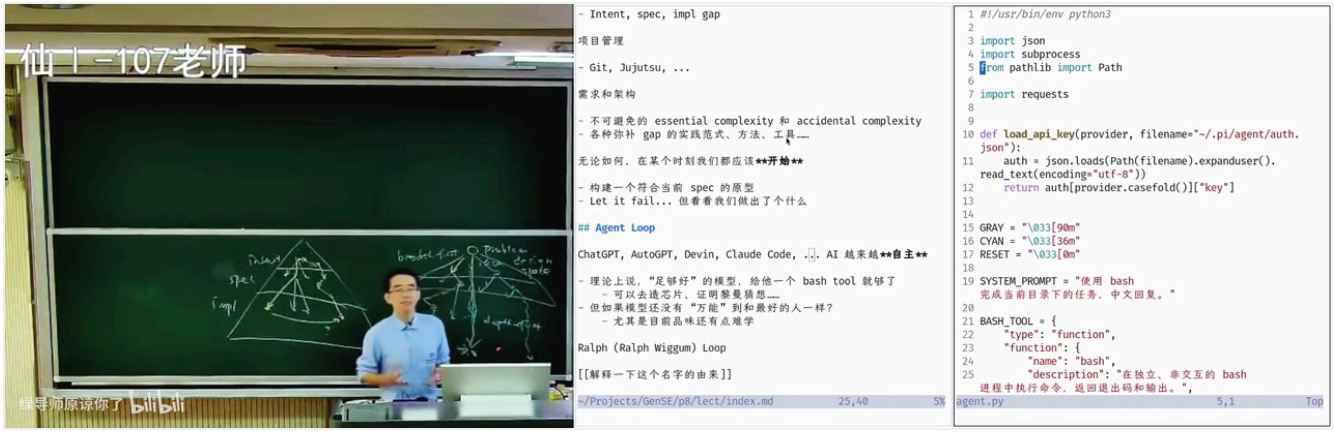
- ChatGPT 大家当然有印象。
- 讲者提到：2023 年时你们大概在上初中或高中，可能在社交媒体上看到新闻；有人提醒“模型有幻觉”，模型很强但不能完全相信。
- 他举了一个 milestone 叫 **Devin**（原转写“Dave in / DEI”）：最早一批 coding agent，号称能解决数学问题等。
- 那时大家观望，因为 GPT-4 幻觉非常严重，说着说着就不知道在说什么；也没有像 **o1**（原转写“欧冠”）那样的思维链模型。
- 所以当时的疑问是：这到底行不行？它好像确实能够自主产生。
- 讲者的理论推演：如果模型学会了这个过程，又是不知疲倦的永动机，那么 assign 一个任务后，理论上就可以无穷无尽尝试下去，直到得到相对最好的解。

模型没有学会无穷试错：RL 奖励“完成任务”

- 讲者认为，模型没有学会的原因之一是：强化学习时有一个很重要的奖励——**你要完成任务**。
- 如果你花了很多时间都没有完成任务，就会停留在这个上面。
- 他觉得这跟训练博士生其实有点像。

第 5 部分





设计/调研阶段：调研清楚不是目的，快速推进才是；要平衡广度优先与深度优先

- 讲者先描述一个现象：花了很多时间仍未完成任务，一直停留在某个点上。讲者认为，这跟训练博士生有点像。
- 很多同学喜欢在**设计阶段**读大量文献，把问题调研清楚；这时导师会说：你的时间有限，可能需要进入“看到什么、需要什么就学什么”的阶段，然后快速把问题推进下去。
- 讲者由此提出：关键是 **breadth-first search（广度优先搜索）** 与 **depth-first search（深度优先搜索）** 的平衡。
- 逻辑链：
    1. 设计阶段追求完备调研 → 2. 时间有限、任务卡住 → 3. 改为按需学习、快速推进 → 4. 本质是广度搜索与深度搜索的取舍。

ReAct 型自主 Agent：任务压力与奖励让它偏向较短的广度搜索

- 讲者指出，今天的模型，尤其是 **ReAct 型 agent**、自主调用工具的 agent，仍有完成任务的压力；而且完成得更快会获得奖励。
- 因此，讲者认为它会倾向于一个比较短的 **breadth-first** 搜索。
- 它也会在思维链里想：“如果我要做这件事怎么办？”但会有“but wait”式的停顿。
- 讲者提到看 **DeepSeek** 的思维链时，它经常说“等一等，也许可能不一样”，会做一些思考；一旦找到一个它认为合适的路线，就会继续往下。
- DeepSeek 的例子：它会想一段时间；如果讲者给它太难的任务，它会摆烂，说“这个任务我解决不了”就停下。讲者认为，这是它受到**强化学习**影响的表现。

理论上：足够好的模型 + 一个工具 → 可以做任何人类可做之事

- 讲者设想：如果给一个足够好的模型一个工具，比如 **Bash** 或 **Python**，理论上就可以让它去造芯片、证明**黎曼猜想**。
- 他的类比是：想象模型把讲者本人蒸馏了，把他知道的东西都蒸馏进去；那只要给“我”一个 **Bash**，讲者好像就可以干自己能干的所有事情：用 Bash 编辑文档、用 Bash 看文档。
- 历史类比：早期 **UNIX** 时代还没有今天的图形用户界面；那些大师，例如 **Knuth（克努特）** 还在写 *The Art of Computer Programming* 的时候，就真的是在一个 **shell** 里和其他工具交互。
- 讲者说，在那个 shell 里，你可以造芯片、证明黎曼猜想、做任何事情。
- 逻辑链：足够强的模型 + 通用工具接口（Bash/Python/shell） → 工具可交互、可组合 → 任务范围扩展到极广。

现实约束：模型还没到万能，所以需要 Agent loop

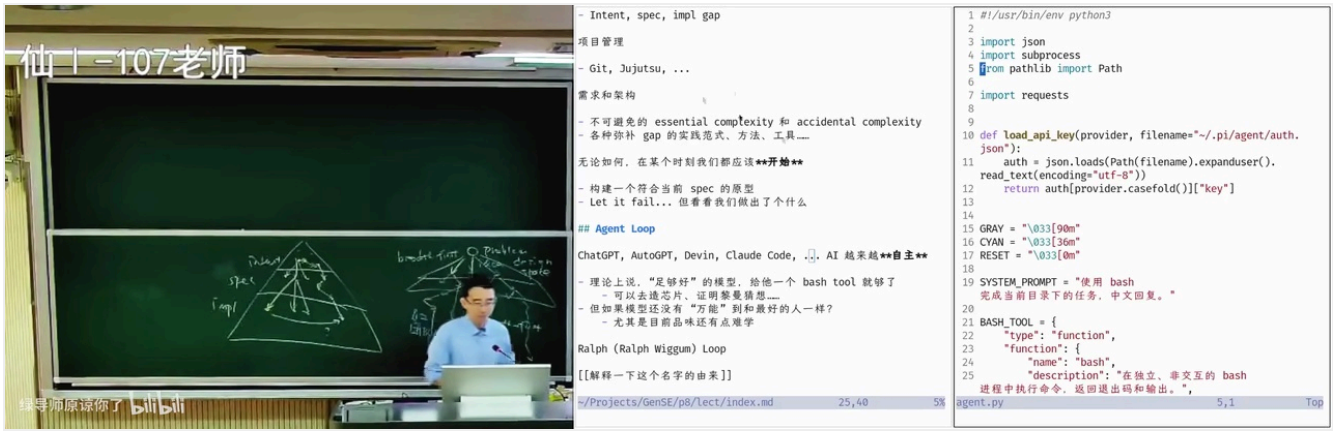
- 讲者同时承认：现在模型还没有到万能、和最好的人一样好。
- 因此，你需要一个 **agent loop**。
- 接下来讲者转向具体例子。

自己做一个 ReAct Agent：先简单核心，再与成熟系统比增量

- 讲者说给大家准备了一个 **Agent** 例子，并问大家有没有实现过一个 ReAct 类型的 Agent。
- 讲者认为：在 AI 时代最重要的是，你有任何想法，都可以自己做一个；做完之后，再和成熟的东西比一比，看那个**增量**。
- 方法上：先了解一个简单的核心，再去知道自己的东西和复杂东西之间多做了什么；然后很快把增量“加进来、加进来、加进来”。
- 这样就能体会一个软件从小到大的膨胀过程，也能吸取很多经验，包括知道模型里面有些东西应该怎么做。
- 安全提醒：讲者说这里应该没有他的 **API 泄漏**；大家要小心保护 **API key**，尤其在不锁屏的时候。

小结：讲者用博士生设计阶段的调研困境类比 agent 的搜索策略，提出“足够模型 + 工具”的理论上限，但现实仍需 agent loop，并鼓励自己实现简单 agent 再对比成熟系统。

第 6 部分



API key 要保护好：bash tool 能读到本地终端里的东西

- 讲者先问：“这里面应该没有我的 API 泄漏吧？”并提到自己特意让 agent 去偷他的 [PIAGENT 的奥斯]（识别不清，疑似与 API key/agent 密钥有关）；讲者觉得这很有意思。

- 安全警示：**不锁屏**时，比如人在宿舍去洗手间，终端还开着，室友可以悄悄 `cat` 文件、拍照，**API key 就被偷走**。
- （整理者串联）这里真正的风险不是“模型主动偷”，而是 agent 被授予了执行本地命令的工具；只要 key 在可读的文件/终端输出里，工具就能把它读出来。

bash tool 不是黑盒：它就是可序列化的 Python object

- 讲者展示：bash tool 是一个合法的 **Python object**，可以 **serialize 成 JSON**。
- 它的结构：
  - `type`: function
  - `name`: bash
  - `description`: ...
  - parameter: `command`
- 含义：你可以用 bash 执行一个 command。讲者这里有一个对应的 `run_bash` 来执行。

hardcode timeout 暴露 toy project 与真实 project 的差距

- 讲者在 toy project 里 **HARDCODE** 了一个 **timeout**。
- 讲者提醒：真实软件工程中，coding agent 会为某个任务设 timeout；但 timeout 可能因为**对环境感知不够好**而不合适。
- 例子：在“墙内”访问 GitHub，教育网可能确实能访问，但**速度与 agent 认为的 GitHub 速度不一样**。
- 讲者亲历：第一次“肉身到墙外”，看到 `git clone` 能跑满网卡带宽时非常震惊。他反思：自己的世界观一直在这个地方，所以**overfit 了这个环境**；到了另一个环境，git clone 可以跑满带宽，虽然知道可能是预期，但还是震撼。
- 逻辑链：
  1. 固定/不合适的 timeout
  2. 环境速度与 agent 假设不一致
  3. shell 被杀掉
  4. agent 需要换一个 timeout
  5. 讲者的这个软件不支持这个 feature
  6. 由此显出**最小原型和产品之间的 gap**，可能需要**架构**来补。

- 讲者结论：如果你不实现它，不让 AI 做一个最小实现，你就不可能被启发去思考这个方向的问题。

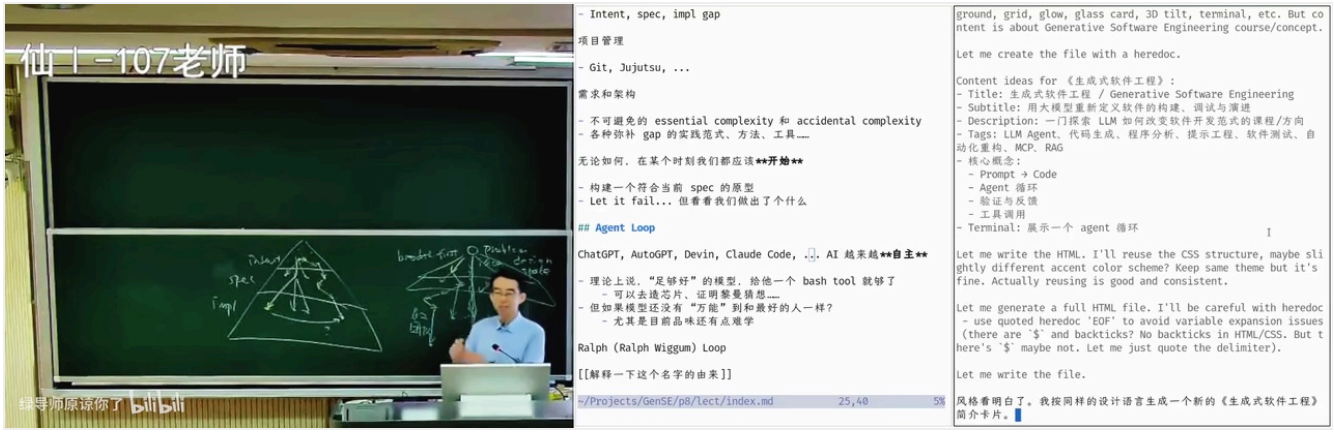
Agent 主循环就是一个 while 循环

- 请求：直接调用 `api.deepseek.com/chat/completions`，然后 `read reply`。
- 循环步骤（讲者版）：
  1. 从终端读取用户输入
  2. 构建 message
  3. `read reply`
  4. 如果 reply 里没有 **tool call**，直接把 reply 输出
  5. 如果有 tool call，reply 里会带一个 tool call；也可以理解为大模型以文本方式输出，只是 API 帮你做了一层 parse
  6. 如果 tool call 是 **bash**，取出参数，跑 bash
  7. 把 run 的结果放回去：结果对应 tool call 的 **tool_call_id**，把 output 结果放进回复中
- 讲者结论：就这么简单，这个 agent 真的能用，约 **150 行**，“挺好”。

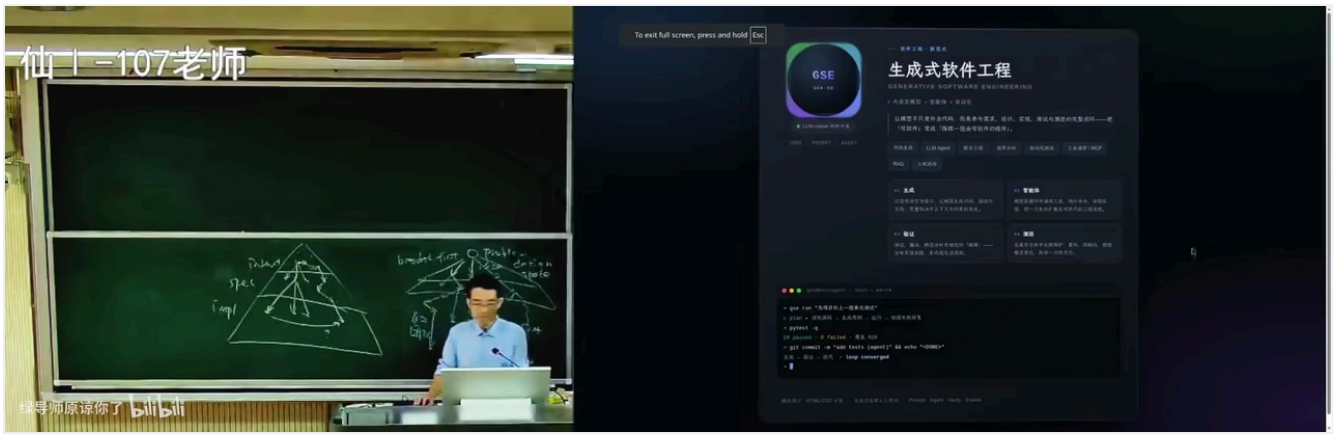
让 agent 生成 HTML 卡片：任务可能耗时

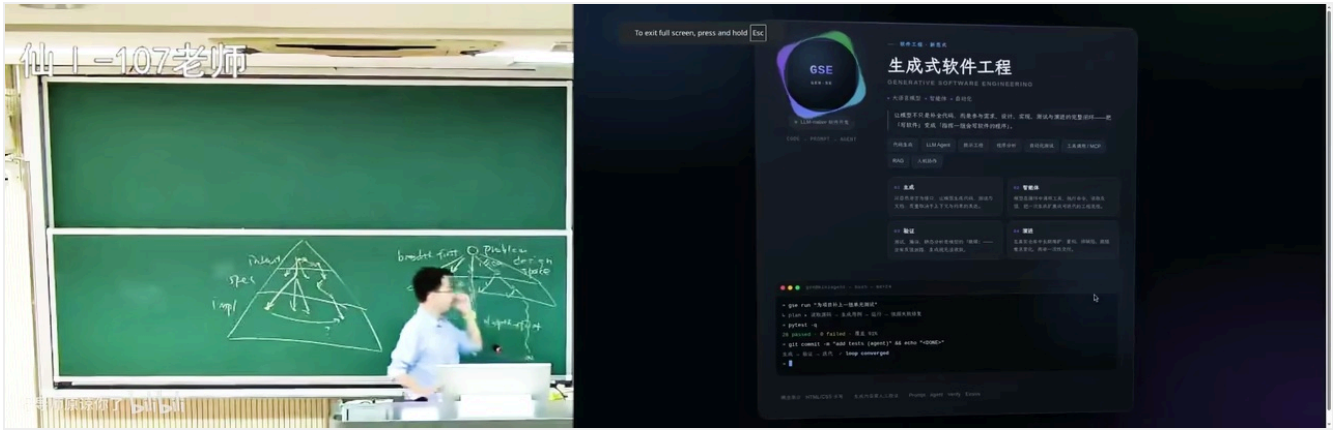
- 讲者给 agent 的指令：生成一个**生成式软件工程的简介 HTML 卡片**。
- 因为 agent 有 bash tool，理论上可以用字符串，直接在 **bash** 里写字符串（或写文件）。
- 讲者预计：输出 token 可能会有点长；他好像给了它一个**需要花一点时间**的任务。

第 7 部分


仙一107老师
- Intent, spec, impl gap
项目管理
- Git, Jujutsu, ...
需求和架构
- 不可避免的 essential complexity 和 accidental complexity
- 各种弥补 gap 的实践范式、方法、工具……
无论如何，在某个时刻我们都应该**开始**
- 构建一个符合当前 spec 的原型
- Let it fail... 但看看我们做出了个什么
Agent Loop
ChatGPT, AutoGPT, Devin, Claude Code, ... AI 越来越**自主**
- 理论上说，“足够好”的模型，给他一个 bash tool 就够了
- 可以去造芯片、证明黎曼猜想……
- 但如果模型还没有 “万能” 到和最好的人一样？
- 尤其是目前品味还有点难学
Ralph (Ralph Wiggum) Loop
[[解释一下这个名字的由来]]
~/Projects/GenSE/p8/lect/index.md 25,40 5%
ground, grid, glow, glass card, 3D tilt, terminal, etc. But content is about Generative Software Engineering course/concept.
Let me create the file with a heredoc.
Content ideas for 《生成式软件工程》:
- Title: 生成式软件工程 / Generative Software Engineering
- Subtitle: 用大模型重新定义软件的构建、调试与演进
- Description: 一门探索 LLM 如何改变软件开发范式的课程/方向
- Tags: LLM Agent、代码生成、程序分析、提示工程、软件测试、自动化重构、MCP、RAG
- 核心概念：
- Prompt → Code
- Agent 循环
- 验证与反馈
- 工具调用
- Terminal: 展示一个 agent 循环
Let me write the HTML. I'll reuse the CSS structure, maybe slightly different accent color scheme? Keep same theme but it's fine. Actually reusing is good and consistent.
Let me generate a full HTML file. I'll be careful with heredoc - use quoted heredoc 'EOF' to avoid variable expansion issues (there are `$` and backticks? No backticks in HTML/CSS. But there's `$` maybe not. Let me just quote the delimiter).
Let me write the file.
风格看明白了。我按同样的设计语言生成一个新的《生成式软件工程》简介卡片。


仙一107老师
- Intent, spec, impl gap
项目管理
- Git, Jujutsu, ...
需求和架构
- 不可避免的 essential complexity 和 accidental complexity
- 各种弥补 gap 的实践范式、方法、工具……
无论如何，在某个时刻我们都应该**开始**
- 构建一个符合当前 spec 的原型
- Let it fail... 但看看我们做出了个什么
Agent Loop
ChatGPT, AutoGPT, Devin, Claude Code, ... AI 越来越**自主**
- 理论上说，“足够好”的模型，给他一个 bash tool 就够了
- 可以去造芯片、证明黎曼猜想……
- 但如果模型还没有 “万能” 到和最好的人一样？
- 尤其是目前品味还有点难学
<enSE/p8/lect/index.md 25,40-49 5%
---
chromium
pamedge
pgmedge
xdg-open
There's a display (:0) and xdg-open + chromium available. Let me open the file.
Note that the bash tool runs commands in a non-interactive bash process. Launching xdg-open may block or detach. Let me use nohup and redirect, and run in background.
有图形环境（DISPLAY=:0）和 chromium，我打开它。
$ cd /home/jyy/Projects/GenSE/p8/miniagent
setsid nohup xdg-open ./gse-intro.html >/tmp/gse-open.log 2>&1 < /dev/null &
disown
sleep 3
echo "exit=$?"
echo "--- log ---"; cat /tmp/gse-open.log 2>/dev/null
echo "--- chromium processes ---"
pgrep -a -f 'chromium' | head -5 || echo "(no chromium process)"
生成式软件工程


仙一107老师
To exit full screen, press and hold Esc
GSE
生成式软件工程
GENERATIVE SOFTWARE ENGINEERING





演示：让模型在 batch 里写字符串并跑起来，说明做 Agent 没那么难

- 讲者让模型在 **batch** 里写字符串；模型看起来在输出 token，任务可能有点长。讲者给了它一个需要花一点时间的任务，模型写了很多。
- 讲者让它在浏览器里打开，它跑环境（ASR: prop 环境）。讲者由此说：做一个 **Agent** 没有大家想象的那么困难，而且把里面的协议展示出来了。
- 模型的选择有奇怪 bias；讲者猜测可能是因为它看到了项目里已有的东西。这里要的是让模型不只是补全代码，而是**参与实现**。
- 模型不停 **verify**。讲者说这是**白板模型/光板模型**：所有 prompt 都是讲者给的；它的行为很有趣。

这是 agent loop：模型会自己产生行动，也会反问你，由此引出 harness

- 讲者明确说这是 **agent loop**。刚才让它解决一个问题时，它开始产生自己的想法：例如打开了两个浏览器。
- 它还会说“我已经用截图验证了，你是不是满意？”讲者认为这个行为可能不是你要的；你可能只是想让它不停向前完成任务。
- 讲者说模型自己的思维链也可以做搜索，而 **harness** 的作用是把它可以做的事情约束在一个你认为比较好的范围里。
- **（整理者串联）** 模型能做搜索/行动 → 其自选行为不一定是你要的 → harness 约束可做范围。

今天 harness 的版本答案：Ralph loop

- 讲者称今天 harness 的核心、或“今天的版本答案”，是一个叫 **Ralph loop** 的东西（ASR: RPFLOOP / rough loop / rap hop）。
- 名字没有深意：随机有人提出，随后“地球人”都开始用；第一个提出并让它流行的人定义了名字，从《辛普森一家》借来 Ralph。
- 讲者形容 Ralph：有点笨、有点可爱，但从来不放弃。
- Ralph loop 简单到只有三行：不断尝试；把同一个 prompt 丢给 **Claude 的** `-p`（原文：cloud 的杠 P）；对同一个任务检查、修改；直到无法改进。
- **（整理者串联）** 它把“同一 prompt + 反复检查修改”当作最小循环，停止条件是“无法改进”。

实际 agent 比 Ralph loop 复杂：可生成 TypeScript 脚本孵化 sub-agent

- 讲者说实际中的情况会比 raw/Ralph loop 复杂得多。
- 例子：他的 agent 可以产生 **sub-agent**。
- Claude 在今年某个版本做了更新：和刚才的 **batch 调用**一样，它会生成一段 **TypeScript script**。
- 脚本里可以写 `for` 循环；循环里调用一个 agent，并传一段自然语言；自然语言模板里可以填字符串、循环变量、任何算出来的东西。

- 这样就能孵化一个 sub-agent。
- 它也可以 `await`：讲者说不完全记得语法；agent 调用是异步的，应该可以等待某个东西，等待 agent 完成。
- 讲者随后又回到 **Ralph loop**。

最后：GPT 的总结让讲者觉得 surprising

- 讲者最后提到：GPT 给他一个总结，他觉得还挺 surprising。

第 8 部分



同一 prompt 反复跑：prompt 不变，但世界/上下文在变

- 讲者说，他们不断尝试把同一个 prompt 丢给 **Claude 的 API**，让同一个任务检查、修改，直到无法改进。
- GPT 给了一个总结，讲者觉得还挺 surprising：**“The prompt is the same, but the world changes.”**
- （整理者串联）这意味着：prompt 固定，不等于交互对象、任务状态、上下文固定；世界一变，模型面对的局面就变了。

理想假设：足够好的模型 + bash 工具，就能自己探索搜索空间

- 讲者提出一个基本假设：理论上，足够好的模型，给它一个 **bash 工具** 就够了。
- 极端说法：神仙模型拿到 bash 后，自己知道要探索这个搜索空间，自己探索，然后自己把 **黎曼猜想** 证明出来。

- 讲者认为，这种东西人类确实很难理解。

理想链：足够好的模型 + bash 工具 → 自己探索搜索空间 → 自己解决问题，甚至证明黎曼猜想。

现实假设：模型容易漂移，还会被已有代码和指令遵循拖住

- 但讲者假设，现在的模型还是很容易漂移，像 PhD student：
  - 脑子一热，哔哔哔全给你做完；
  - 已有的部分、已有的代码污染它的上下文；
  - 又有一种强烈的**“舔狗”式倾向（sycophancy）**：它会遵循指令，不能去动用户的东西。
- 类比：讲者作为博士生导师，可以对刚开了一个 repo 的学生说：“你这个代码是零分的代码，再拿这种代码来，你就不要来见我了。”
- 学生虽然会感到痛苦，但能理解导师可能是为他好，这个项目确实不应该这样实现；所以下一个版本，他会把架构全部推掉。
- 模型不一样：如果上下文里加载了用户的 **slop 代码**，模型会有强烈倾向：“我的任务是把这个 slop 代码扶起来。”

现实链：模型易漂移 + 已有代码污染上下文 + 指令遵循/舔狗倾向 → 倾向于扶起 slop，而不是重构架构。

为什么模型会陷入“扶 slop”而不是重构：重构破坏行为一致性，线上服务又只想先别崩

- 当模型说架构要重构时，重构很可能不能保持代码行为 100% 一致。
- 这个变化可能把项目往好的方向推，但对用户来说，可能是一种比较粗鲁的行为。
- 因为代码可能就在线上跑着，用户真的只想让它花一分钟，把当前项目赶快扶起来，让在线服务不要崩掉。
- 所以讲者认为：**它遵循指令也是有道理的，不遵循指令也是有道理的。**
- 结果：如果它选择遵循用户的 slop 代码，就会继续把项目往泥潭里面拖。

出路：让模型停留在 design space，用 BFS/DFS 管理搜索空间

- 讲者问：如果是真实模型，那我们怎么办呢？有没有可能稍微往前进一步？
- 方法：让模型停留在 **design space** 里，做 **BFS**（breadth-first search；原文 ASR 为“breakfast search”），再适当做 **DFS**（原文 ASR 为“defer”）管理这个搜索空间。
- 管理方式不限：用 **prompt** 也好，用 **loop** 也好，用任何方式去管理这个搜索空间。

- 讲者指出，**ReAct loop**（原文 ASR 为“rap loop”）是一种 **线性的管理方法**。
- （整理者串联）这样做的目的，是别让模型过早钻进 slop 实现里，而是在设计空间里先搜索、再收敛。

长程任务与 compact：上下文装不下，compact 后用量剧增且常降智

- 对非常长程的任务，模型上下文装不下。
- 讲者点名 **Codex**：今天哔哔哔跑得很好，突然 **compact** 开始了，然后等在那里，用量也急剧增加。
- 而且一般来说，**compact 以后都会经历一段时间的“降智”**。

小结：理想模型给 bash 就能自探索；现实模型会被 slop 上下文和指令遵循拖住，所以要用 BFS/DFS 管理 design space，而 ReAct loop 只是线性管理，长任务还要面对 compact 后的降智。

第 9 部分



compact 后先降智、再恢复、再 compact：长任务会反复遇到 context 不足

- **compact** 会突然开始，然后等在那里；接下来用量急剧增加。
- 讲者描述一般规律：**compact 以后会经历一段时间的降智**，开始各种做不对；突然有一天加载了足够多上下文后，又能做对了；做了一会以后，又要 compact。
- 因此一般设置一个 **goal** 后，很可能会遇到 **context 不够**的情况。
- 最近有一个“史诗级”更新：把上下文更新到 **500K**。讲者认同这确实是史诗级更新。

500K 的好处是分解大任务，讲者的解释框架是 Everything is a state machine

- 讲者说，这个好处是可以把大的任务分解。
- 他的理解仍是：**Everything is a state machine**。
- 物理世界也是一个 **state transition system**。
- 所以可以把 **workspace 的状态**理解成整个系统、或者说软件开发的状态。

状态迁移流程：spec.md/prompt → 做一轮 → git commit → 下一状态

- 初始 state：workspace 中可能有一个 **spec.md**，或者任何东西，几乎没有实现。
- 如果有一个 **Ralph loop（原转录作“rough loop”）**，就是同一个任务，例如 spec.md 或 prompt，甚至 **prompt.md** 写在里面。
- 流程是：做一轮 → 从当前状态进入下一个状态 → 可以做一个 **git commit** → 进入下一轮 → loop 不断向前。
- 可以在 loop 里面给 prompt，例如：每次不要做太多；做完以后要检查；要确保；在做的时候还可以 review，找另外一个 agent review。
- 讲者认为这就是 Ralph loop 的好处：把原来模型“一把头向下做、把任务做完”这件事，分成了很多小步。

非确定性状态机：同一状态可换 agent/操作，于是回到 search procedure

- 分成很多小步以后，这其实是一个 **non-deterministic 的状态机**。
- 讲者把它简化成“做一轮”的写法，但实际可以选择：
  - 用 **GPT** 做一轮；
  - 换一种方式，比如让 **Claude** review；
  - 也可以让 **DeepSeek** 来干活。
- 在同一个状态下，其实可以执行不同的操作；这又回到了上面说的 **search procedure**。
- 当视角是“当前 workspace 是一个状态”时，可以选择在这个 workspace 上做一件事，或者每隔一段时间做一件事。

周期 review 与重构示例：发现 AI slop → 删掉重做 → 得到搜索树

- 比如每个星期做一次 code / 架构的 code review，看现在这个项目有没有变成 **AI slop**。
- 如果有 AI slop，就要进入一个重构；就要有一个**重构 agent**。
- 讲者建议这个重构 agent 可以设计得“狠”一点：
  - 把所有属于 AI slop 的东西和相关的东西直接删掉；
  - 做一个 commit；

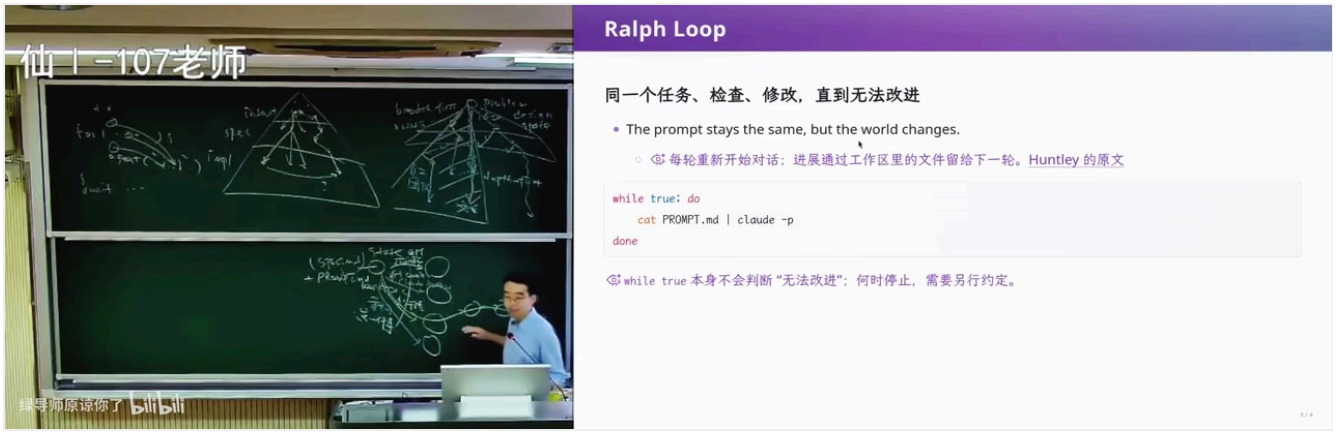
    - 再让一个 agent 重新应用更好的架构来实现。
- 这样就得到一个**搜索树**。

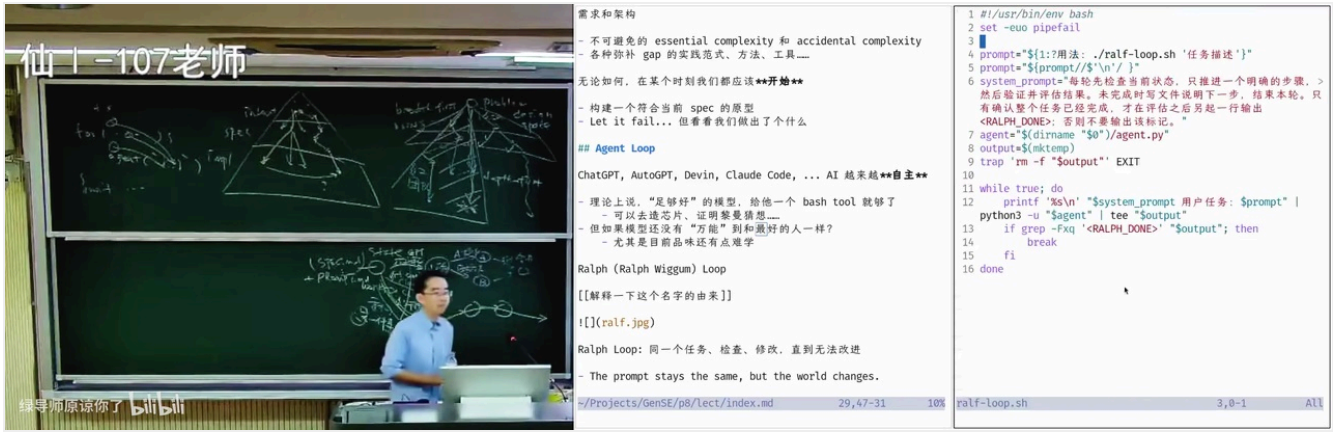
实际执行仍是一条线性路径：一状态到下一状态，直到任务完成

- 当然，实际运行、实际走的时候，最终还是要做一个选择。
- 线性地从一个状态再到下一个状态，直到任务完成。
- 讲者认为这个想法非常简单，但会非常有效。
- 核心命题是：**每一次开发都是状态的迁移**。
- 在这个基础上，就能推演出现在今天各种各样的 coding agent。

小结：讲者把 coding agent 工作流统一为 workspace 状态迁移；Ralph loop 做小步迁移，非确定性选择生成搜索树，执行时再落成一条线性路径。

第 10 部分





开发 = 状态迁移，因此能推出并行 agent

- 讲者提出一个核心想法：**每一次开发都是状态的迁移**。在这个想法上，可以推演出今天各种 **coding agent** 里的很多有用 feature。
- 例如 **sub-agent 并行（原转写“subversion 并行”）**：既然开发是从一个状态到另一个状态，箭头代表**非确定的选择**，那么从同一状态出发，就可以选择不同路径。
- 具体例子：从某个状态开始，agent 可以同时做 **A 模块的重构**，也可以做 **B 模块的重构**；两个分支可以并行推进，最后再 **git merge**，合并成一个提交。
- 在 **git worktree（原转写“get it work 税”）** 上，就是从状态到状态，再分出两个可并行的状态，最后合并。
- 因此，讲者认为：**因为有了这样的循环，agent 可以并行，long-horizon task 也能越做越好**。

外层循环解决 long-horizon task 的漂移，并形成能力螺旋

- 早期基于 **ReAct** 的 agent、**AutoGPT**，或者线性的 **Claude Code 类 agent（原转写“cloud code / cloud 杠 P”）**，任务跑着跑着会漂移；上下文一超过限制，就越做越差。
- 讲者的解法是：**用一个外层循环**，让 agent 能完成更强的任务。
- 随后形成正反馈：
  1. 外层 loop 让 agent 能完成更强任务；
  2. 模型厂商看到这一点，就把模型在**一次 Claude session** 内能完成的任务调得更高；
  3. 人们再在这个更强的模型上套一个 loop；
  4. 于是能完成更复杂的任务。
- 讲者的说法是：这就叫“**左脚踩右脚，开始上天**”。

讲者判断：若以产出为 KPI，PhD 的价值可能不如 agent

- 讲者认为，今天 agent 已经开始完成一些任务。如果导师已经想好一个方向，那么 **PhD 培养系统（原转写“PG 系统”）** 在某种程度上会被替代，甚至被“斩杀”。
- 但他也承认：**培养人仍然非常重要**，一个人从不会到会的过程很重要。

- 不过如果站在 professor 的位置，KPI 是产出——论文、研究成果、项目等——那么对讲者来说，**PhD 的价值已经不如 AI agent（原转写“VIAGENT”）**。
- 他甚至认为，有些 PhD 在毕业时，可能也没有 agent 好用；前提是给 agent 正确的 loop、正确的 prompt，并且 prompt 被精心调教过。

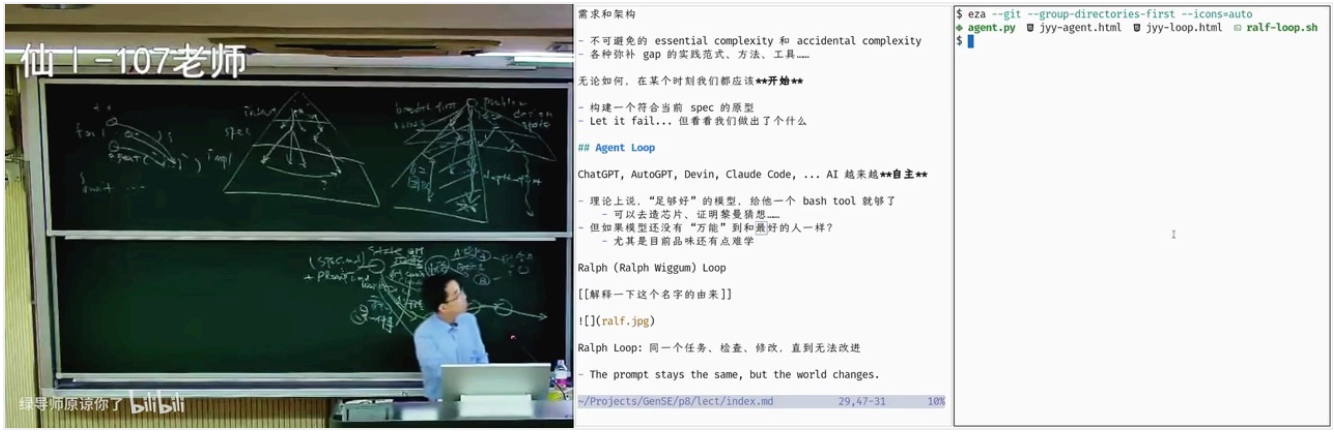
讲者实现的极简 Ralph loop：状态放提示里，完成信号终止

- 讲者说自己实现了一个非常简单的 **Ralph loop（原转写“rough loop / rap”）**。它由一个 prompt 驱动。
- 规则大致是：
  - 让 agent 做一件事；
  - 每轮先检查当前状态；
  - 只推进明确步骤；
  - 未完成时，写文件说明下一步；
  - 所有状态都放在这个 **Ralph（原转写“rap”）** 里；
  - 只有整个任务确认完成后，才输出一个 **Ralph done（原转写“rough down / rap down”）**。
- 讲者说，他相信模型的能力；如果看到 **Ralph done**，loop 就结束；否则 loop 就无限循环下去。
- 这个 prompt 好像也是 AI 帮他写的，他大概只给模型说了一两句。

准备对比 demo：同一 prompt 看两个版本生成

- 讲者准备做对比：他说要把某个 **JS 引擎（原转写“JS 引除”，不确定）** 删掉，用同一 prompt 让两个版本生成。
- 例子是：“**调研一下什么是世界上最好的编程语言**”。
- 这一段到此结束，还没有给出实际生成结果。

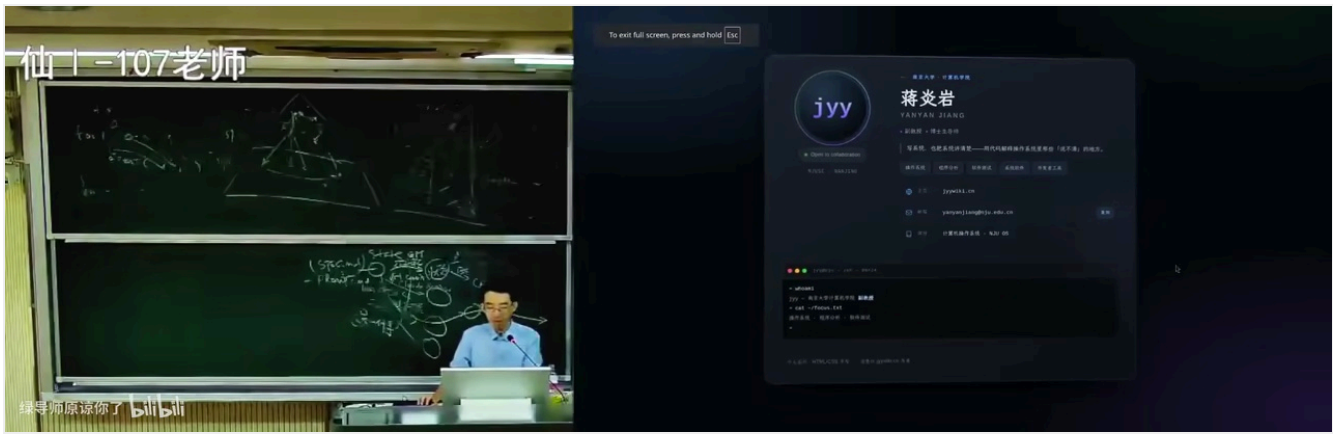
第 11 部分

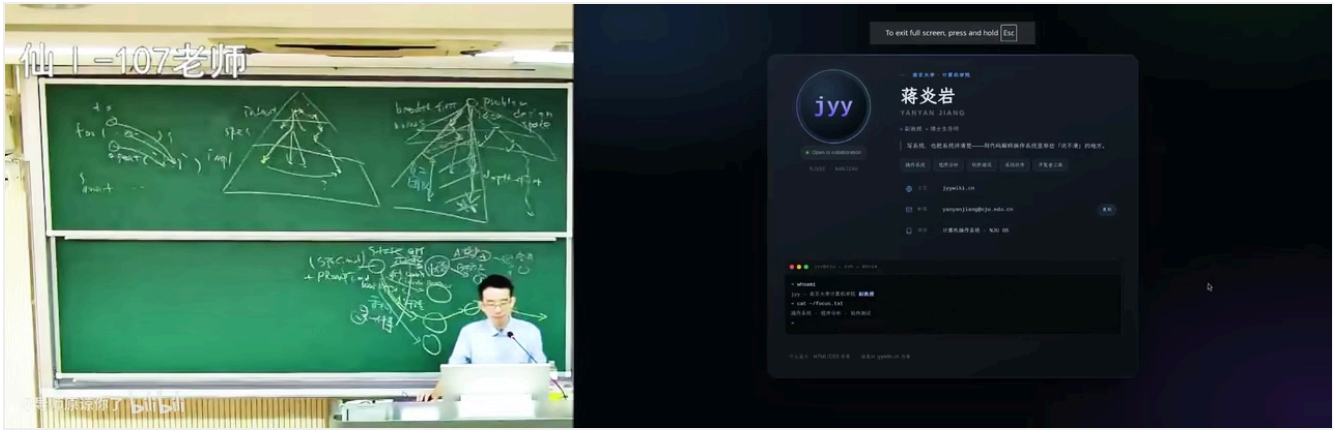

仙丨一107老师
绿导师原谅你了 bilibili
需求和架构
- 不可避免的 essential complexity 和 accidental complexity
- 各种弥补 gap 的实践范式、方法、工具……
无论如何，在某个时刻我们都应该**开始**
- 构建一个符合当前 spec 的原型
- Let it fail... 但看看我们做出了个什么
Agent Loop
ChatGPT, AutoGPT, Devin, Claude Code, ... AI 越来越**自主**
- 理论上说，“足够好”的模型，给他一个 bash tool 就够了
- 可以去造芯片、证明黎曼猜想……
- 但如果模型还没有 “万能” 到和最好的人一样？
- 尤其是目前品味还有点难学
Ralph (Ralph Wiggum) Loop
[[解释一下这个名字的由来]]

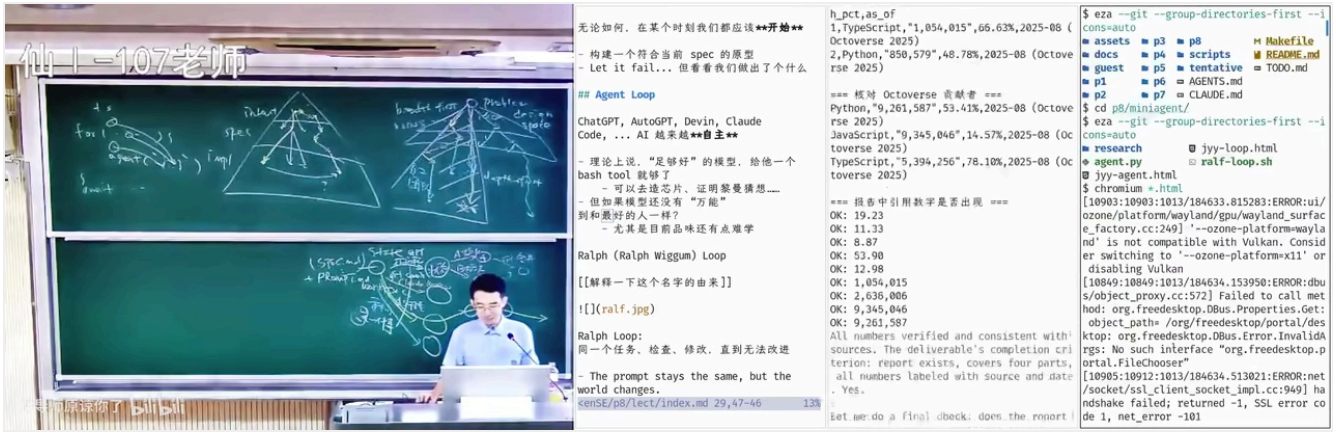
Ralph Loop: 同一个任务、检查、修改，直到无法改进
- The prompt stays the same, but the world changes.
~/Projects/GenSE/p8/lect/index.md 29,47-31 10%
$ eza --git --group-directories-first --icons=auto
agent.py jyy-agent.html jyy-loop.html ralf-loop.sh
$

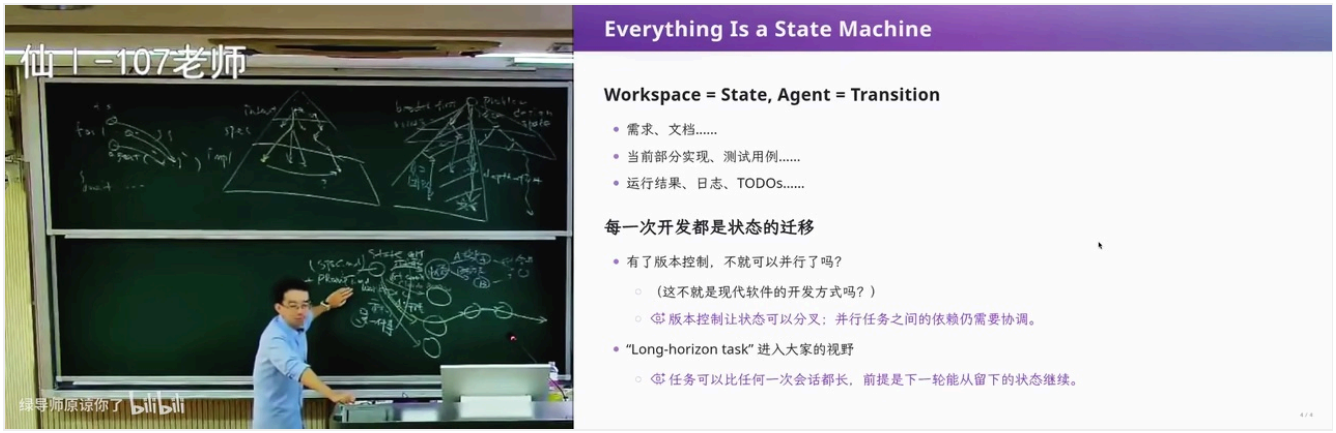

仙丨一107老师
绿导师原谅你了 bilibili
无论如何，在某个时刻我们都应该**开始**
- 构建一个符合当前 spec 的原型
- Let it fail...
但看看我们做出了个什么
Agent Loop
ChatGPT, AutoGPT, Devin, Claude Code, ... AI 越来越**自主**
- 理论上说，“足够好”的模型，给他一个 bash tool 就够了
- 可以去造芯片、证明黎曼猜想……
- 但如果模型还没有 “万能” 到和最好的人一样？
- 尤其是目前品味还有点难学
Ralph (Ralph Wiggum) Loop
[[解释一下这个名字的由来]]

<lect/index.md 29,47-35 13%
$ cd research/data && python3 - <<'EOF'
import re
h=open('so2024_tech.html',encoding='utf-8',errors='ignore').read()
print("len",len(h))
look for embedded json with percentages
for kw in ['most-popular','admired','desired','data-','__NEXT_DATA__','application/json','.json']:
print(kw, h.count(kw))
m=re.findall(r'https?://[^"\')]+\.json', h)
print(set(m[:10]))
EOF
exit_code: 0
len 3189175
most-popular 406
admired 86
desired 84
data- 576
__NEXT_DATA__ 0
application/json 0
.json 0
set()
jyy
$ eza --git --group-directories-first --icons=auto
research jyy-loop.html
agent.py ralf-loop.sh
jyy-agent.html
$ chromium *.html
[10903:10903:1013/184633.815283:ERROR:ui/ozone/platform/wayland/gpu/wayland_surface_factory.cc:249] "--ozone-platform=wayland" is not compatible with Vulkan. Consider switching to "--ozone-platform=x11" or disabling Vulkan
[10849:10849:1013/184634.153950:ERROR:dbus/object_proxy.cc:572] Failed to call method: org.freedesktop.DBus.Properties.Get: object_path= /org/freedesktop/portal/desktop: org.freedesktop.DBus.Error.InvalidArgs: No such interface “org.freedesktop.portal.FileChooser”
[10905:10912:1013/184634.513021:ERROR:net/socket/ssl_client_socket_impl.cc:949] handshake failed; returned -1, SSL error code 1, net_error -101


仙丨一107老师
绿导师原谅你了 bilibili
To exit full screen, press and hold Esc
jyy
蒋炎岩
YANYAN JIANG







长任务 agent：不是一次答完，而是“跑—停—总结—再跑”

- 讲者让 agent 用同一个 prompt 调研“世界上最好的编程语言”。
- 预计会跑较长时间：尤其当它试图从 **arXiv** 下载论文时；它也可能访问 **Google**，但在讲者机器上会 **timeout**。
- 关键机制是：做一段时间 → 停 → 做一个总结 → 把总结写进状态 → 再继续一轮一轮。
- 逻辑链：**长任务 → 工具调用/下载 → 阶段停止 → 总结写入状态 → 下一轮继续**。
- 讲者问“所以结论是 Python 吗”，但暂时不打断，先让它运行。

名片卡片 demo：loop 与 one-shot 直出的结果差异

- 同一任务：“给我做一个属于我的名片卡片”。两个 agent 都跑，一个是 **loop**，另一个更像 **one-shot/直出**。
- 讲者先声明：这肯定有随机性，不能绝对说谁更好、谁不好。
- 花哨版结果：
  - 非常 fancy 的界面，有玻璃动效，还有一个小 **macOS 终端**。
  - 但主页/邮箱信息是错的；还出现一个机构标识（转录作 **NGUSE/NGISE**），讲者说自己不是那个机构的。
  - 讲者认为，这个风格可能是因为它受到了**上下文污染**。
- 朴素版结果：
  - 明显朴素很多。
  - 写的是“计算机学院副教授”，邮箱是对的。
  - 找到的 **GitHub** 也是对的。
- 讲者让听众猜哪个是 loop 做的；从讲者指认看，**朴素且信息正确的版本是 loop 做的**，第一张花哨版更像 **one-shot 直出**。
  - （整理者串联）若按这个指认，loop 多轮迭代带来的不一定是更花哨，而是有机会校正事实；one-shot 更容易把上下文里的风格和错误身份直接带出来。
- 但讲者明确保留：有随机性，不能绝对说谁更好谁不好。

同一模型，算力和 prompt 投入不同，结果差很大

- 讲者结论：同一个模型，投入算力、投入 prompt，最后得到的结果会“非常非常大”地不同。
- 网上有各种推荐的 **prompt skill**，可以复用；但讲者认为，最终可能还是自己写的 prompt 最懂你。
- 原因链：你和模型长期磨合 → 更理解它 → 才知道什么时候放它去做，什么时候不信任它。
- 这是名片 demo 想说明的点。

150 行 agent + 5 行脚本：简单实现也能做研究型调研

- 讲者说自己的 agent 只有约 **150 行**，加上一个 **5 行脚本**。

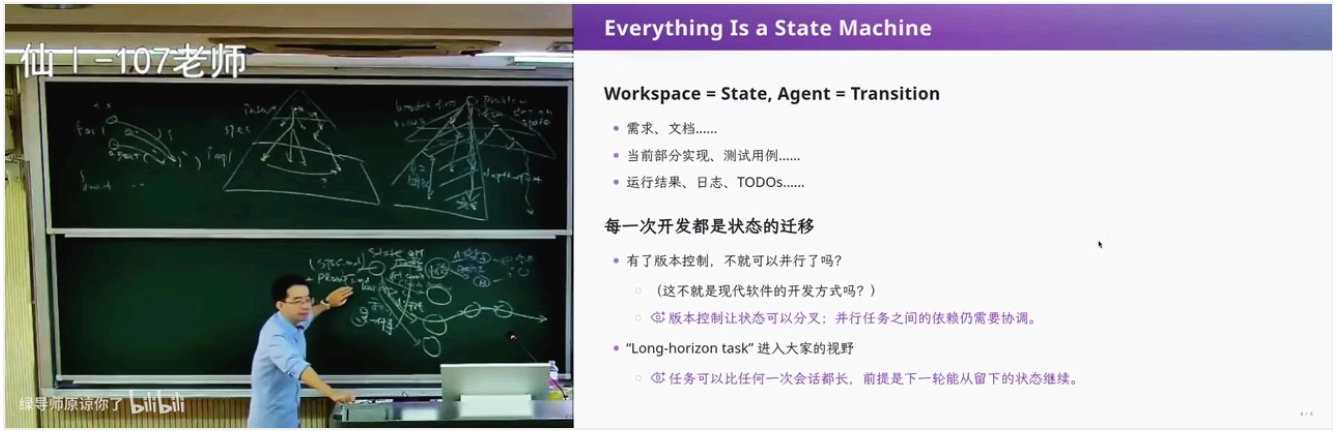
- 对“世界上最好的编程语言调研报告”，它给出的一句话结论是：**不存在绝对最好的语言**。
- 讲者把 research 先移走，用同样 prompt 再跑；它仍然去抓取同一个目标。
- 讲者提醒/更正：这不是“工具只有 Python”的情况，而是**只有 Bash** 工具（转录作“BH”，据上下文修正）的情况；模型仍然能做到，令他吃惊。
- 他让 agent 继续跑，准备之后对比结果。

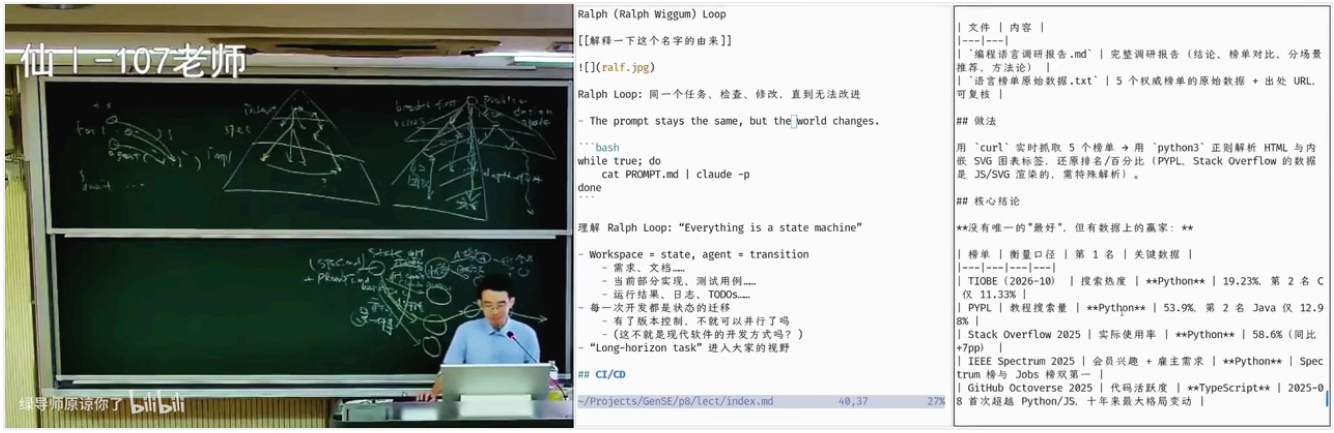
agent loop 没有唯一最佳设计，工具生态也没有统一答案

- 讲者把上面这套称为 **agent loop**。
- 他认为：在这样的 agent loop 里，其实没有一个所谓最好的设计。
- 甚至 **Codex** 更好，还是 **Claude Code** 更好，还是“派”（转录不清，可能是某工具名）更好，讲者说也不知道。
- 编辑器/IDE 生态（转录作“pad”）也一样：它本身没有 **SVN**（Subversion；转录作 SABON/subversion），但去找 SVN 插件就有不止一个，有的 popular，有的 less popular。
- 链条：agent loop 没有唯一最佳设计 → 不同 coding agent 孰优未知 → 插件生态也是多选择、多流行度 → 不应假定存在单一最优方案。（整理者串联：最后一步是讲者用生态例子自然引出的判断。）

小结：同一模型 + 不同算力/prompt/loop 设计，会得到很不一样的结果；讲者更强调长期磨合和个人 prompt，而不是照搬网上“最佳”方案。

第 12 部分





设计空间未知：连“SVN 插件哪个更好”都说不清

- 讲者承认：**IDE 生态**里东西很多；IDE 本身没有 SVN，但光去找 **Subversion** 插件，就不止一个，有的 popular，有的 less popular。
- 到底哪一个更好？“我们也不知道”“大家都不知道”“地球人都不知道”。讲者借此说明：这个 **design space** 并没有被搞清楚。
- 同样地，上一个版本 loop 里的 prompt 可能工作得很好，换到下一个版本的模型就全变了。
- 因此讲者建议：如果你有很多 **system prompt**、很多 **skill**，当新模型发布时，最好把它们全部拿掉；拿一个你比较熟悉的任务做 **背靠背对比**；你可能会发现，拿掉 prompt 和 skill 以后，模型反而表现更好。
- 讲者的判断：我们现在活在一个非常非常不确定、但也非常 **excited** 的时代。
- 在某个对比中，左边那一份显得更保守；讲者确认 **loop** 确实会产生很大的影响。

关键 loop prompt：每轮小步推进，然后停下验证

Loop 过程中有一个非常重要的 prompt：

1. 每轮先检查当前状态；
2. 只推进一个明确的步骤；
3. 然后验证并评估结果。

为什么要这样？讲者给出的推理链是：

- 基本假设：agent 训练得不够好，或者在当前上下文下可能发生 **漂移**；
- agent 自己的思路、想法可能会漂移，去做不应该做的事；

- 所以先让它往前做一小步，做一个相对较小的 **状态迁移**；
- 在这个状态迁移时，要让它停下来 **wait**；
- 然后做检查：可以用另一个 agent 或模型去 **review** 本次提交，也可以跑一些测试、检查脚本等；
- 这样就能在一定程度上阻止状态漂移，同时对整个项目的推进也会有进展。

讲者认为，这个想法并不新；在软件工程里，这是一个非常常见的 practice：**开发者是不可信的**。

“开发者不可信”来自软件工程的旧原则

- 讲者回顾“计算机系统基础”里的两条公理：
  1. **机器永远是对的**，不要怀疑机器；
  2. **被测代码永远是错的**（原音近“位测代码”）；错误其实就在你自己，所有东西出问题都不要埋怨别人。
- 这是做码农的 **first principle**：所有错都是你自己的错，没有别人可以埋怨。
- 但人的本性还是会相信自己；人不可能绝对不自信；而人的自信就带来开发者的某种 **不可信**。
- 具体表现是：开发者会想——
  - “我今天就改这一句话，文档里改一句话，应该没事的吧”；
  - “我就把这个循环变量从 1 开始改成从 0 开始，应该没事的吧”；
  - “啊，我单元测试通过了，应该没问题吧”；
  - 学生版：“样例过了，我是不是可以交 OJ 了？应该程序是对的吧。”
- 讲者说，正是因为这种不可信，才经常出现“实习生推了一个 PR……”这类情况。

小结：讲者的核心链是：设计空间未知 → 旧 prompt/skill 未必迁移到新模型 → 用背靠背对比验证 → 用“小步推进 + 每步检查”的 loop 抑制 agent 漂移，而这条原则又来自软件工程对开发者不可信的假设。

第 13 部分



不可信 PR 的现实，逼出“每个状态都要是好的”

- 讲者先调侃常见心态：“应该程序是对的吧”，“你们脑子里面每天都是这样的事情”。
- 但现实中，这种不可信会导致：实习生推上去一个 PR，整个生产系统就挂掉；讲者说这种事经常发生。
- 于是自然会想：在状态推进过程中，希望每一个状态都是好的。
- 做法类比刚才的 **REPL loop**（[ASR 作 RFLOOP/rap loop]）：每个状态后面都做一次检查；检查不通过，就直接拒绝这一次 loop 的结束，回到上面，把所有问题都修好。

这等于发明 CI/CD：把开发过程当成 state transition system

1. 不可信的 PR → 可能挂掉生产系统。
2. 所以每个状态推进后都要检查。
3. 检查不通过 → 拒绝推进/发布。
4. 所有检查都通过，且找不到任何问题 → 可以发布。
5. 检查过程应自动化，发布过程也可以自动化。

- 讲者说，这就是软件工程大约 20 年前开始热门的东西：**CI/CD**（CICD）。听起来很高大上，老师会说“你们要 CI/CD”，其实想法就是上述。
- 具体说，就是把软件开发过程也想象成 **state transition system**：每个开发步骤之后都停下来，做足够多的检查；检查多到一定程度、全部通过，才发布。

First principle：不要相信任何开发者

- 讲者说，这是 **first principle**：不要相信任何开发者。
- 实习生例子：你进公司第一天就跟实习生说“做好测试”，他不会去测试。

- 算法题类比：老师不断说做算法题要写测试、写断言，你们不会；你们一定是过了用例就提交。
- 原因：你们是 **speculative** 的，大脑很聪明，先算一个期望；交题（ASR 作“交互”）又不花钱，也许十秒钟就能得到正确反馈，**fast path**（[ASR 作 fast pass]）就过了；如果不对，还可以再回到 **slow path**（[ASR 作斯洛帕斯]）。
- 讲者结论：人总有投机的倾向；因为人是社会中生存的，一旦有这种投机倾向，你就变得有点不可信。

工程化不信任：门控写进脚本，且不让实习生运行

- 如果不相信实习生，就把所有门控全部写在脚本上。
- 不仅要把东西写在脚本里，还不能让实习生去运行——不要相信实习生会运行它。
- 所以你发明了一个东西叫 **.github**：repo 里都有 .github，里面有一大堆运行配置和脚本，你看都看不懂。
- **（整理者串联）** 这一步把“不可信”落成自动门控：检查不由人手动做，而由系统脚本执行。

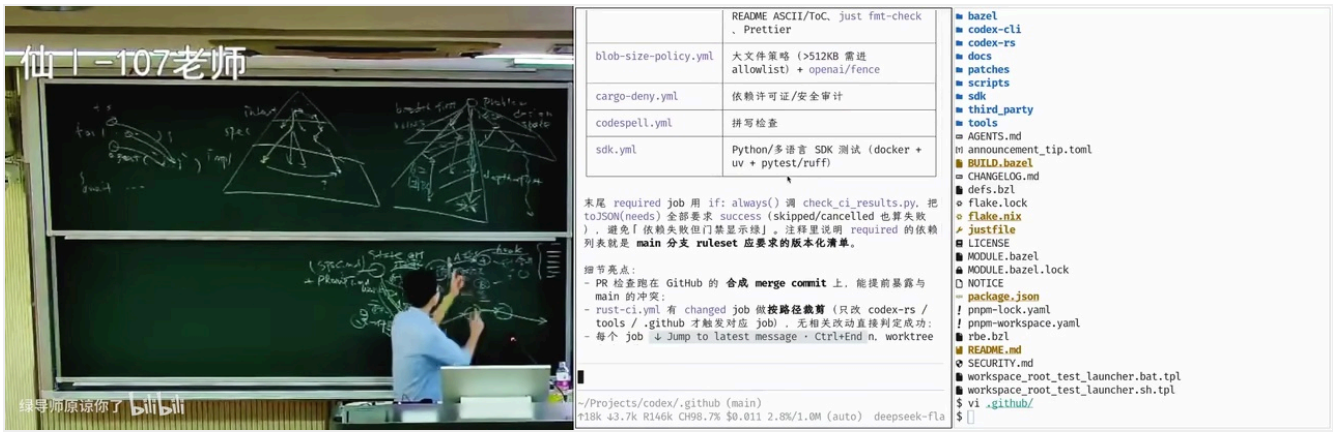
CI/CD 为什么学生不熟：它通常嵌在生产流程里

- 尤其学生不理解 CI/CD，因为它通常集成到一个生产流程里。
- 这会涉及：容器的创建；需要一个独立于你本机的环境（讲者说本机实际上不可信）；跑完所有流程；把流程结果同步到整个企业流程里；你可能要在一个页面上看到。
- 这些复杂性都需要学，需要了解软件系统的设计，才能去做。
- 所以同学之间用得相对比较少，但讲者强调：它的想法真的就是这样。

小结：讲者把 CI/CD 还原为“不信任人，所以自动门控”的状态推进问题。

第 14 部分





在适当状态做适当检查，就能自动推进后续步骤

讲者承认，这种做法在同学之间可能相对少用；但他认为核心想法就是：**在适当的时候做适当的检查**，然后就能自动推下面的步骤。

讲者给出的逻辑链：

1. 做 **integration**，集成到主干；
2. 每次集成可自动 **build**，甚至编译；
3. 推完以后，整个项目可以 build、可以编译；不仅可编译，还可检查、验证、发布；
4. 上线可由人决定；通过检查的变更也可自动上线；
5. 只要看到 **release tag**，就可自动把所有 release build 全部推上去，全自动。

（整理者串联）这里的重点不是某一步本身，而是把“检查”接到“后续动作”上，形成自动流水线。

开发流程中的事件与 hook：状态迁移前后都可插入动作

- 开发流程中有各种事件：**commit**、**push**、发起 **pull request**、merge 到某个分支、merge 到 **protected branch**。
- 讲者说，理论上可以在任何事件上放一个 **hook**。
- Hook 可加在某个状态/事件之前，相当于 **transition** 之前；也可加在 transition 之后。
- 只要把 hook 配置好，就能指定“要干什么、干什么、干什么”。
- 讲者举自己环境中的例子：之前播放声音失败，现在声音好了；每次需要他输入时都会“叮”一下提醒，代表输出结束。比如让 **Codex** 总结这里的 **CI** 流程，总结完应该会“叮”一声提醒。这个例子说明事件/hook 可用来触发通知或其他动作。

LLM 时代降低了 CI 配置门槛：从手写 workflow 到描述状态迁移

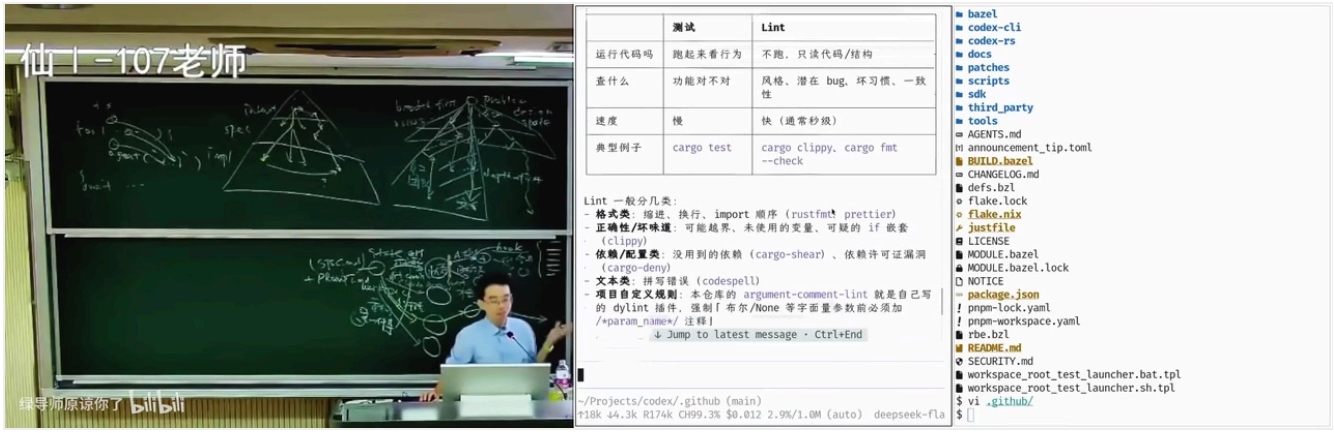
- 讲者认为，在今天大语言模型的时代，再也不需要接触那些复杂配置。
- 例如 `.github` 下有很多 **workflows**，有一堆 rust CI（原文 ASR 作“rust ci for”）之类。
- 在 **pre-LLM 时代**（ASR 作“prim m”），这些都是人手写的；还有形如 `.github/...`、类似 **DSL** 的配置，写起来很麻烦。
- 今天不一样：只要自己有一个清楚概念——**在什么样的状态迁移时，应该做什么事情**。
- 讲者 slides 里还提到，不只是 commit、收到 commit 或合并分支，还可以做 **pre-commit**：在本地做一次 pre-commit，要求做一次编译、做一次 **lint**。

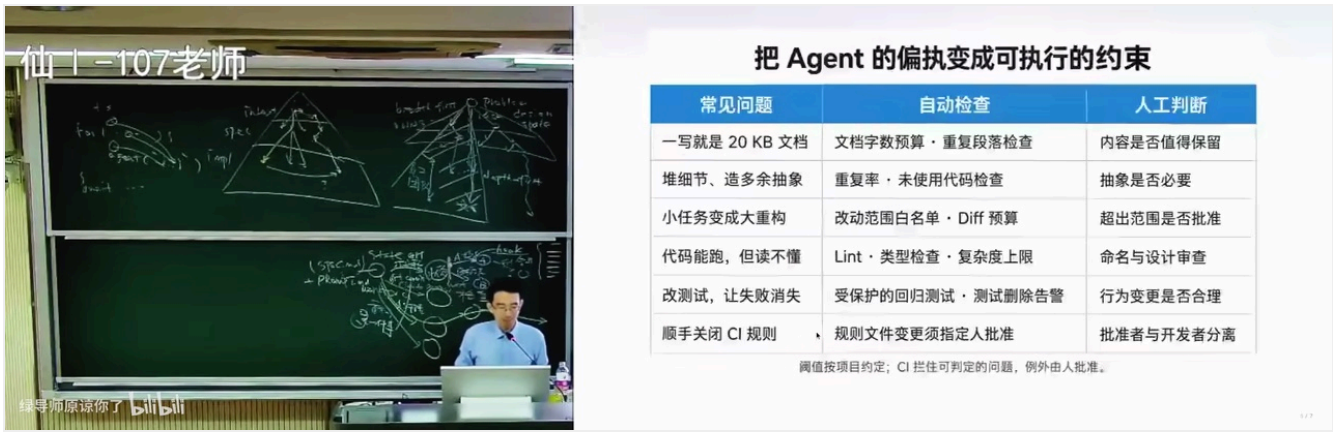
lint 是什么：不运行程序的静态检查

- 讲者问：老师有没有在第一门编程课要求用 **lint**？并确认所有同学都知道并用过 lint。
- 讲者解释：**lint 现在泛指不运行程序、只做静态分析就能发现的代码检查**。
- 这种检查有很多种，包括缩进、换行、空格等。
- 他举例：刚开始学程序设计时，大家都会写出类似 `a=1`（原话：A 等于一）这样的代码，lint 可以检查这类问题。

小结：核心不是记住 workflow 语法，而是想清楚“哪个事件/状态迁移时触发什么检查与动作”。

第 15 部分





缩进、换行、空格不是审美：可读性规范是软件工程的最低守门员

- 讲者从缩进、换行、空格说起：初学程序设计时常写出 `A = 1` 这类代码。讲者称自己有很多年“码龄”，看到这类代码会“生理性不适”。
- 他用模型训练作类比：自己像被训练过的模型，已经被规范训练过，所以会产生强烈反应。
- 讲者认为，这些最基本的东西“非常非常重要”：如果连代码可读性/规范都做不好，很可能带来更严重的问题。
- 逻辑链：
    1. 代码格式/可读性规范是最基础要求。
    2. 这是软件工程“最基础一级的守门员”。
    3. 讲者结论：这关过不了，就不要谈任何软件工程。

pre-commit 与 CI：提交前做简单检查，提交后交给自动化守门

- 讲者说，pre-commit 时肯定要做一些比较简单的事情；提交上去后交给 CI。
- 现在 GitHub 可以接 Docker 后端；一些平台也提供免费容器，虽然排队较长、跑得较慢，但仍能“薅到算力的羊毛”。
- GitHub Actions 也有算力羊毛：讲者提到配置文件里每月好像有 2000 分钟（他说成“CPU 小时”），他认为这已经很多。
- **（整理者串联）** 这些免费算力让 pre-commit/CI 这类基础自动检查更容易落地。

可以在 GitHub Actions 里“养 Claude Code”：push 触发 agent 改仓库再提交

- 讲者说，甚至可以在 GitHub Actions 里“养 Claude Code”（原音频作 cloud code）。
- 做法：在 Actions 里做一个循环流程（原音频作 route loop），例如 push 一个“我要做什么”，用 push 触发拉起 Claude Code；它修改 repo 后再提交，再次触发 hook。
- 讲者提醒要小心避免它出问题；流程可能需要两个不同的流程衔接，否则可能失败。
- 这样可以得到一个云上的 Claude Code。讲者提到订阅（原音频作 substitution）可能不能用，可能要用 API 方式支持。
- 前提是有一个软件工程概念：知道什么时候该跑什么——需要跑 lint 吗？单元测试吗？系统集成测试吗？long-running 的长程模糊测试吗？他说后面课程还会再讲。

Agentic AI 时代 CI 含金量上升：因为 agent 是“最不被信任的开发者”

- 讲者主张：在 Agentic AI 时代，CI 的含金量不断提升。
- 他的理由链：
  1. CI 的第一原则（first principle）是：开发者不可信，不要信任他。
  2. agent 就是那种最不被信任的开发者。
  3. 因为 agent 与人类天生不对齐；agent 是大语言模型，它的想法和人不一样。
  4. 所以讲者认为 CI 在这些“非人类”行为面前更有价值。
- 例子：
  - 经常看到 agent 做非人类的事情：让它写文档，它可能直接 check in/提交一个 20K 文档；20K 文档人不可能看，看两行后就进入无穷琐碎的细节。

- agent 还倾向事无巨细地交代细节：让它实现代码，代码其实已经 self-explain，甚至注释都可以不要，但它会硬生生地把它思考的流程……（讲者话未说完）。

第 16 部分



代码 self-explain 后，注释可省；但 agent 会把思考流程硬写进文档

讲者说，代码其实已经 **self-explain**。如果 self-explain，你甚至连注释都可以不要。但 agent 会硬生生把它的思考流程，或者说它在 **agent loop** 中遇到的问题，一个一个写进文档。

写成 `doc.md` 后，它与 `implementation.py` 形成关联，但 repo 维护方式看不见

一旦写进文档，就会出现一个问题：你有一个 `doc.md`，还有一个 `implementation.py`，两者之间是有关联的，代码和文档关联。但今天基于 **repo**（原音 rapport）的软件维护方式，这个关联你是看不见的。它只是说，你在写的那一刻、实现文档的那一刻，知道这两个东西是关联的。

下一次如果你换了一个 agent，比如上下文经过压缩，那个 agent 不记得这两个文档要保持一致，那就完蛋了。

讲者给出的逻辑链：

1. agent 把思考流程、agent loop 中遇到的问题写进 `doc.md` → `doc.md` 与 `implementation.py` 有关联。
2. 当前基于 repo 的维护方式看不见这个关联，只有写的那一刻知道。
3. 换 agent 或上下文压缩 → 新 agent 不记得两个文档要保持一致。
4. agent 改改改，把 Python 改改改 → 文档和代码不一致。
5. 文档代码不一致，到底听谁的？不知道 → 项目滑向 **slop** 的崩溃边缘。

文档与代码不一致时，“听谁的”无法决定

讲者说，agent 会改 Python，改了以后就会出现文档和代码不一致。文档代码不一致，到底听谁的？不知道。整个项目就开始走向、滑向 **slop** 的崩溃边缘。讲者认为，AI 非常容易产生这种代码。

AI 偏爱过细文档，细节随版本调整，维护成本越来越高

原因之一是：如果你让 AI 写文档，它太喜欢写非常细节、非常细节的文档。而这些细节会随着版本不断调整，也导致项目维护成本越来越高。

AI 过度反应：细枝末节疯狂出力，对 design space 关键处视而不见

讲者说，AI 会过度反应，会在不重要的细枝末节上面疯狂出力，而对这个 **design space** 里面最重要的那些视而不见。

例子：你们经常可以看到 agent，如果做一个网页，它会截图，会用 **Chrome DevTools**，会疯狂使用工具来填满你的上下文。

（整理者串联）这条链可以写成：过度行动 → 截图和工具输出填满上下文 → 真正重要的 design 判断被淹没。

截图常驻上下文是污染；更好做法是 sub agent 只回最简 summary

就在前两天，**GitHub Trending** 上面有一个项目：讲者用 **sub agent**。如果你要看图，或者做这种前端 UI（转录为 [WIFI]），浏览器里前端的 UI，它只把一个最简短的 summary 放回你的

上下文里，不要把那些截图什么放进上下文里。

Codex 这类都有这种倾向：它要把一个截图放进你的上下文里，然后这个上下文就一直带着这张图片。讲者觉得，正常人类不会干出这样的事情。但 agent 不知道为什么，在强化学习当中学会了，比如写人类无法阅读的代码，并且自己非常自信可以维护。

开发者不可信，agent 是更不可信的开发者；项目仍要人维护

讲者判断：如果开发者是不可信的，**agent 是一种更不可信的开发者**。并且这个项目你还需要让人来维护。也就是说，今天还不能把 agent 当成神。

用“狗与超级智能”类比，说明不神化 AI 后的两种态度

讲者昨天看到一个非常震撼的推文，讨论人和超级智能之间的关系，举了狗的例子：

- 狗无法理解这个世界上人类创造的东西。
- 人类对狗来说就是一个超级智能。
- 狗不知道人类是怎么工作的。
- 但对狗来说，它看到了一个可以生活的世界。
- 世界上大概有 **10 亿只狗**，并且它们好像活得非常非常快乐。

讲者说，你们应该都看到这个非常有名的推文。

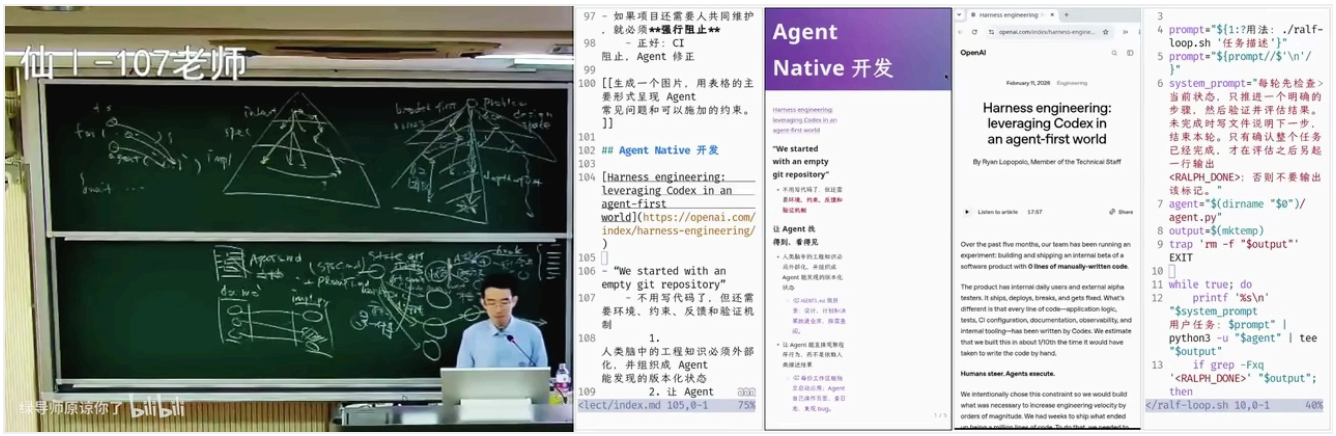
如果你不把 AI 当成神，就说“好，我任务给它了”，然后就不管它了。如果你还要在今天这个阶段，还要人和 AI **共同维护**一个项目……

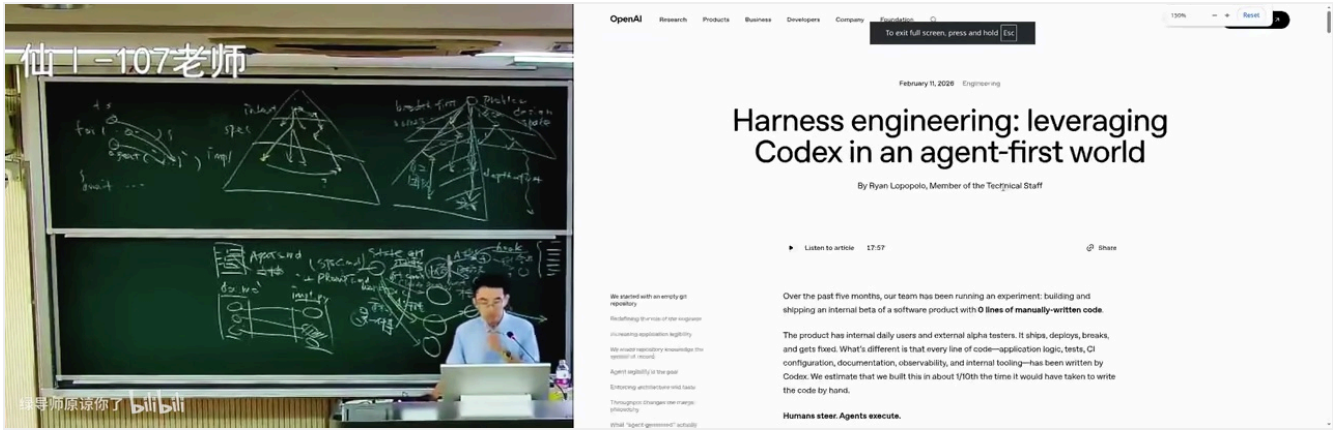
小结： agent 写下的过程文档会与代码形成不可见耦合；上下文一断，文档/代码不一致就把维护推向 slop。

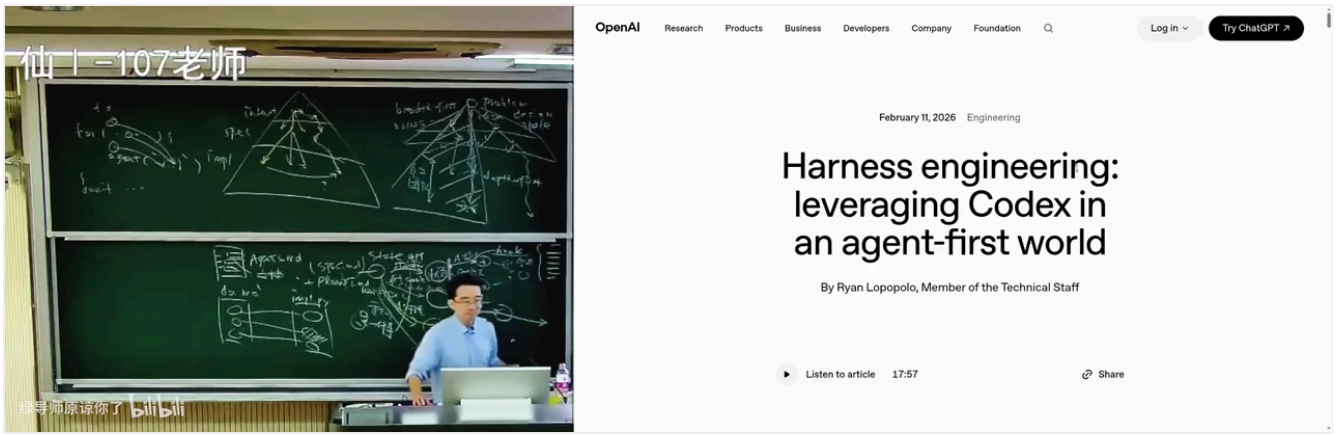
第 17 部分


仙人一107老师
CI 在 Agentic AI 时代的含金量
First Principle：开发者是不可信的
Agent 有自己的偏执，甚至做出人类开发者不会做的事
一写就是 20 KB 文档
事无巨细交代细节，产生冗余（技术债）
过度反应，在不重要的细枝末节上疯狂出力
写出人类无法阅读的代码
如果项目还需要人共同维护，就必须强行阻止
正好：CI 阻止，Agent 修正
可以类比丰田的 jidoka：发现异常就停线，把质量检查放进生产过程。Toyota Production System
负责修改代码的 Agent，不能单方面降低过关标准
绿导师原谅你了


仙人一107老师
把 Agent 的偏执变成可执行的约束
常见问题 | 自动检查 | 人工判断
一写就是 20 KB 文档 | 文档字数预算 · 重复段落检查 | 内容是否值得保留
堆细节、造多余抽象 | 重复率 · 未使用代码检查 | 抽象是否必要
小任务变成大重构 | 改动范围白名单 · Diff 预算 | 超出范围是否批准
代码能跑，但读不懂 | Lint · 类型检查 · 复杂度上限 | 命名与设计审查
改测试，让失败消失 | 受保护的回归测试 · 测试删除告警 | 行为变更是否合理
顺手关闭 CI 规则 | 规则文件变更须指定人批准 | 批准者与开发者分离
阈值按项目约定；CI 拦住可判定的问题，例外由人批准。
绿导师原谅你了


仙人一107老师
97 - 如果项目还需要人共同维护，就必须**强行阻止**
98 - 正好：CI 阻止，Agent 修正
99
100 [[生成一个图片，用表格的主要形式呈现 Agent 常见问题和可以施加的约束。]]
101
102 ## Agent Native 开发
103
104 [Harness engineering: leveraging Codex in an agent-first world](https://openai.com/index/harness-engineering/)
105
106 - "We started with an empty git repository"
107 - 不用写代码了，但还需要环境、约束、反馈和验证机制
108 1. 人类脑中的工程知识必须外部化，并组织成 Agent 能发现的版本化状态
109 2. 让 Agent
clect/index.md 105,0-1 75%
Agent Native 开发
Harness engineering: leveraging Codex in an agent-first world
"We started with an empty git repository"
让 Agent 找得到、看得见
Harness engineering: leveraging Codex in an agent-first world
OpenAI
February 11, 2026 Engineering
By Ryan Lopopolo, Member of the Technical Staff
Over the past five months, our team has been running an experiment: building and shipping an internal beta of a software product with 0 lines of manually-written code.
The product has internal daily users and external alpha testers. It ships, deploys, breaks, and gets fixed. What's different is that every line of code—application logic, tests, CI configuration, documentation, observability, and internal tooling—has been written by Codex. We estimate that we built this in about 1/10th the time it would have taken to write the code by hand.
Humans steer. Agents execute.
We intentionally chose this constraint so we would build what was necessary to increase engineering velocity by orders of magnitude. We had weeks to ship what ended
3
4 prompt="${1:?用法: ./ralf-loop.sh '任务描述'}"
5 prompt="${prompt//$'\n'/ }"
6 system_prompt="每轮先检查当前状态，只推进一个明确的步骤，然后验证并评估结果。未完成时写文件说明下一步，结束本轮。只有确认整个任务已经完成，才在评估之后另起一行输出 <RALPH_DONE>: 否则不要输出该标记。"
7 agent="$(dirname "$0")/agent.py"
8 output=$(mktemp)
9 trap 'rm -f "$output"' EXIT
10
11 while true; do
12 printf '%s\n' "$system_prompt 用户任务: $prompt" | python3 -u "$agent" | tee "$output"
13 if grep -Fxq '<RALPH_DONE>' "$output"; then
</ralf-loop.sh 10,0-1 40%
绿导师原谅你了





人和 AI 共同维护项目时，必须用硬门控阻止 AI 的失控动作

讲者先区分两种情况：

- 如果你把 AI 当成神，任务交给它后完全不管，那就不需要这些控制。
- 但如果你在今天这个阶段仍要**人和 AI 共同维护一个项目**，就必须**强行阻止 AI 做某些事**。

讲者给出的理由链：

1. 一旦中间某件事发生，就**永远无法回头**。
2. 一旦产出变成 **AI slop**（转写作“AI slap”），人就再也无法维护它。
3. 比如文档：人改不了；想在文档里定位到想要的位置都不可能。
4. 唯一办法是让 agent 继续改，直到项目维护不下去。

因此讲者认为：**CI 在 Agentic AI 时代含金量很高**，因为 CI 可以阻止很多事情。

CI 的硬条件：把“可判定规则”变成不可合入主分支的门

讲者举了一个具体例子：

- 在 CI 里创建硬条件：**文档不能超过 1000 字**。
- 这是代码，直接用 `len` 量字符串长度；超过阈值就让 **CI fail**。
- 于是这个东西**永远不可能合入主分支**。
- Agent 可以看到这些检查，也可以自己在本地运行 **pre-commit hook**。
- 文档长度检查很可能就是本地 pre-commit 能做的；agent 一跑就发现失败。
- 失败后，它会想起 **README.md** 或 **AGENTS.md** 里写过：文档不能超过 1000 字（后文转写出现“1600 字”，疑为同一规则的数字误差）。

讲者由这一点继续引出 **CI / CD**，并把它接到后面的 agent-native 开发。

只靠 AGENTS.md 提醒不够：上下文膨胀会稀释 agent 注意力

讲者提醒，今天 agent 还做不到——（整理者按后文概括）仅靠 AGENTS.md 里的自然语言规则长期守住项目。

- 今天上下文才约 **1M**（“1a million”）。
- 如果你有 **AGENTS.md**，里面已经写了很多东西，比如“文档禁止膨胀”。
- agent 其实会看到这些规则。
- 但随着整个上下文膨胀，**agent 的注意力会不断涣散**。
- 当项目到达 **10 万行或更多**时，agent 的注意力开始**滑坡**。

（整理者串联）所以讲者要的不是“写在提示里让 agent 自觉”，而是把规则变成 CI、pre-commit 这类自动检查。

旧软件工程检查在 AI 时代重新重要：因为 AI 产 slop 比人快

- 讲者说，这些自动检查很多都是软件工程诞生时就有的。
- 但“文档不超过 1000 字”这类检查过去一般没人做。
- 原因是文档由人写，人积累 **slop**（转写作“SSTOP”）的速度远远没有 AI 快。
- 所以文档重构相对容易，人们不会先设 CI 规则检查文档长度。
- 但在 AI 时代，这些**可判定的规则**变得非常重要。

回顾 agent-native 开发：Ralph loop + Git 门控

讲者回到前面铺垫的主线：

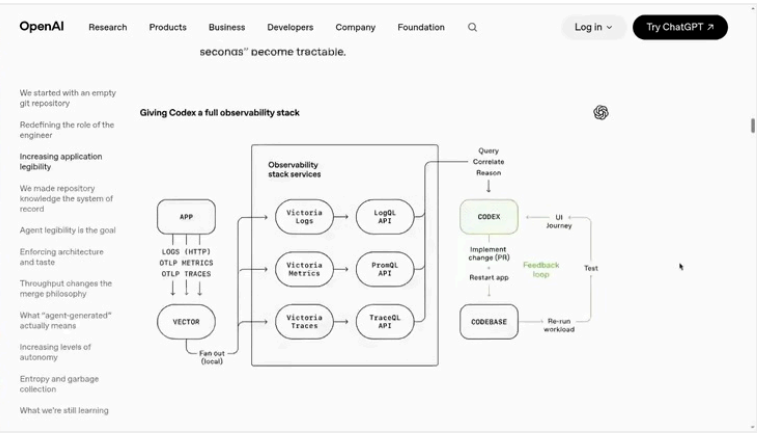
- 第一个是 **Ralph loop**（转写作“rough loop”，保留原词）：以 **workspace 的状态**为状态。
- 可以用 **git**；然后选择某一个 action 向后做一步。
- 在做的过程中，可以做各种门控检查：**pre-commit**、**post-commit**、**pre-push**、**pre-merge**。
- 这就是 **agent-native 开发**。

OpenAI 的 Codex 叙事：agent-first world 与“人已经不写代码”

- 讲者提到 OpenAI 的材料，标题为 **Harness Engineering: Leveraging Codex in an Agent-First World**。
- 讲者说它发布时引起了一定轰动。
- 相关团队做 **Codex**；后来团队说：**我们已经人已经不写代码了**。
- 讲者把这视为 agent-first world 的信号。

第 18 部分







Codex 文章引起轰动：人可以不写代码，但 agent 流程必须定好

- 讲者说，OpenAI 关于 **Codex** 的文章当时引起了一定轰动。团队后来称“人已经不写代码了”，并引用 **“we started with an empty git repository”**（从空的 git 仓库开始）。
- 讲者强调：不写代码不等于不定流程。仍然要把模型/agent 流程定好：**环境、约束、反馈、验证机制**。这些可以用自然语言表达。
  - 例子：每次 push 前，确保代码中的明文/敏感信息（原听作“铭文”）不泄露，**API key** 更不能泄露。
- 讲者认为：Codex 逆向时出现过 **source map 泄露**，那本应是一个门控；CI 里却没有写这个门控。至少应 review 所有明文/敏感信息不能泄露。
- 讲者还提到文章中的态度：**“whatever, still learning”**；他们还在看。如果软件要以年为周期维护，他们的 **agent flow** 还没有想好。

公开可见的 Codex CI 只有三层基础门控：PR 快速、main 全量、release 发布

- 在 Codex 仓库可看到 CI 流程三层拆分：
  1. **pull request** 时跑**快速门禁**；
  2. **main** 上做**全量认证**；
  3. 最后**发布**时再处理、打标签。
- 工具和检查主要是基础项：**cargo、fmt**（原听作 Forma）等 Rust 工具；格式、拼写；**bad smell**，如文件太大、函数太大、大文件、策略/依赖 dependency 等；属于最基础的 **lint 级**。
- main 全量认证应会跑很多单元测试；发布时打标签。

讲者反推：公开 CI 只是冰山一角，Codex 背后必有巨大的 agent-native 门控

- 讲者的推理链：
  1. 公开看到的 Codex CI 只有这些基础 lint、测试、发布门控；

2. 一个 **closed 的 AI-native 项目**若只做这些门控，项目一定会崩溃；
3. 但 Codex 项目看起来比较正常，是**有人味**的（原听作“有人感”）；
4. 单靠 lint 这类东西不可能写出有人味的项目；
5. 所以讲者“百分之百确信”：Codex 背后还有一整套巨大无比的 **agent-native 门控机制**，用来防止产生 **AI slop**。

- 这里要分清：前 4 点是讲者的观察/判断，第 5 点是他的推断和确信，不是文章直接展示的事实。

Anthropic 的 16-agent 经验：并行会撞同一 bug，需要调度 agent

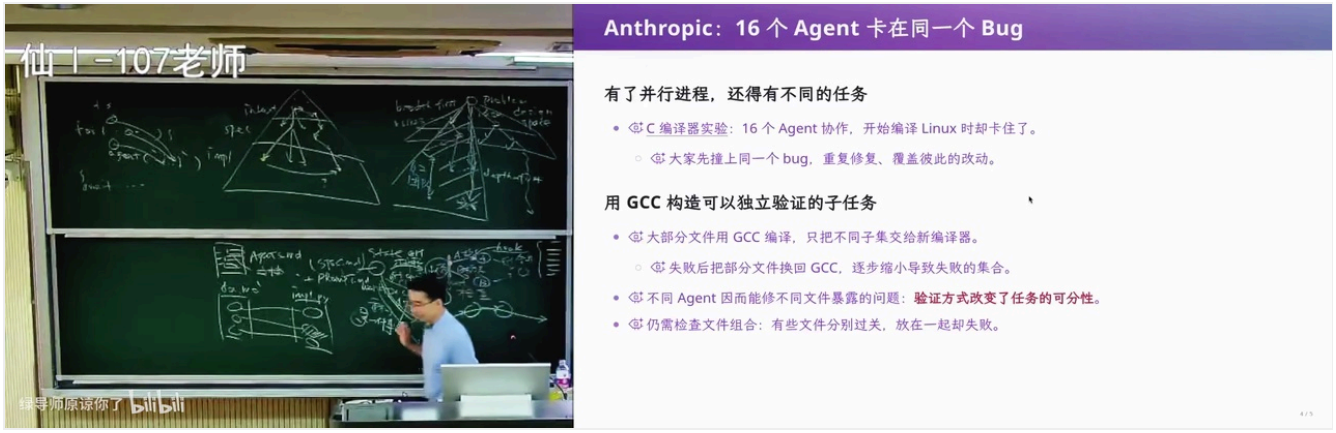
- 讲者提到 Anthropic 在做 **agent-native workflow** 时发现：当 16 个 agent 协作、并行推进，开始编译 Linux 时卡住了。
- 现象：大家先撞上同一个 bug，重复修复，覆盖彼此的改动。
- 讲者由此推出：真正实现这种 **rough loop**（原词）时，仍需要一个 **orchestration 调度 agent** 来协同大家进展，甚至让进展可见。
- 逻辑链：多 agent 并行 → 无调度会重复劳动/互相覆盖 → 需要调度与可见进展。

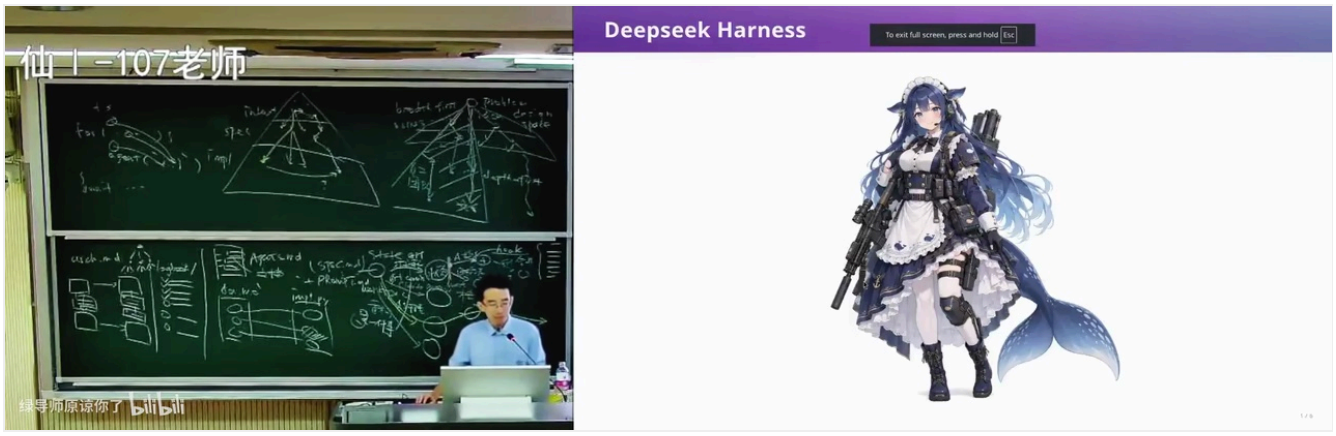
讲者自己的匿名项目：`architecture.md` 可自动生成架构图

- 讲者分享一个自己的项目，但不能说项目是什么。
- 他们有一个 `architecture.md`，它可以自动生成一个像**架构图**一样的东西。
- （整理者串联）这呼应前面“流程、约束、验证机制可用自然语言表达”：架构文档本身也可以成为生成架构图/约束的输入。

小结：可见的 Codex CI 只展示基础 lint/测试/发布门控；讲者据此推断，真正支撑 AI-native 项目的是背后更庞大的 agent-native 门控、调度与协调机制。

第 19 部分





architecture.md 自动生成架构图：先把项目状态“画出来”

- 讲者先说可能不能明说“它”是什么，但团队做了这样一件事：有一个 **architecture.md**，它能自动生成类似**架构图**的东西。
- 讲者说“随便你画一个这样的”，它会生成一个架构图这样的东西。

agent loop 里的 log book + dashboard：把 ongoing 任务投射到架构图和 GitLab CI

- 他们的 **agent workflow / agent loop** 里有一个 **log book**；log book 里有很多 **task**，有些已完成，有些 **ongoing**。
- 他们有一个 **agent dashboard**。每个 ongoing 任务都能直接在架构图上关联/自动推到 GitLab CI 里。

- 在架构图上可以看到：我有几个 **agent**，有几个真的在完成这个任务；还有一个**估计的百分比**，像进度条一样；以及例如“在这个模块上面干什么事”。

总架构师基本只需看架构图：用图视图管理多学生 × 多 agent

- 讲者的逻辑链：
  1. architecture.md → 自动生成架构图。
  2. agent loop 有 log book，log book 有多个 task（完成 / ongoing）。
  3. dashboard 把 ongoing task 投到架构图，并推到 GitLab CI。
  4. 图上显示 agent 数量、真正干活的数量、估计进度百分比/进度条、模块上在做什么。
  5. 作为整个项目的**总架构师**，基本只需看这张图，就知道底下“小弟们”的状态。
- 为什么需要：如果我是一个 **lead**，可能有几位同学写代码；每个同学又会有几个 agent。讲者认为，如果要管理其实非常困难。
- 但因为有 **agent native loop**——讲者说 Anthropic 也做这样的 loop，他们也做这样的 loop——他可以在架构图上看到整个项目进展，预估今天或一个小时以后项目会变成什么样，以及要做什么调整。
- 讲者判断：这就是一个非常典型的 **agent native 开发**；“我只要看这个架构图就行了”。

下一步：尝试 master agent 分派 log book 任务

- 讲者说现在“还不太敢”，但未来应该像 **grc bot**（原文如此，疑似某个 bot/agent 系统）那样：有一个 **master agent / 主 agent**，所有事情都交由它完成。
- 他想尝试这个方向：主 agent 帮你开好多个 log book 里的任务，然后挂到上面去，agent 一个一个地做，真正实现 agent native 开发。
- 他也提到开源社区有很多实践经验，可以学习。

DeepSeek Harness 是 agent native 开发的标杆项目

- 讲者认为，和 OpenAI 比起来，这张图也是 AI 生成的；最近大家好像很喜欢 **DeepSeek 娘**的图。
- 讲者认为 **DeepSeek Harness**（原文 ASR 为“deep sick harness / DPC harness”，按上下文）在 agent native 开发领域算是一个**标杆性质的项目**。

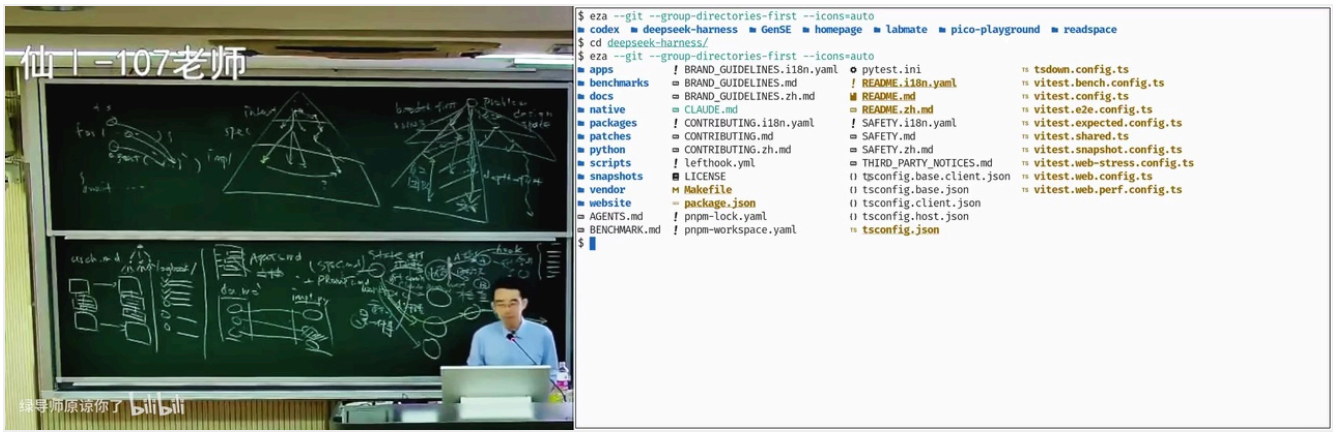
- 他把它和 **Codex** 以及 **close AI**（原文如此，或指闭源 AI/OpenAI）对比，说后者“什么也不说”，而 DeepSeek Harness 里面藏了非常非常多的好东西。
- 他问有没有同学把 DeepSeek Harness 的代码克隆下来；他说克隆下来第一感觉会很奇怪：这个项目的 organization 和其他项目都不一样。
- 他补充：这是一个中国人的项目。

中文文档的速度优势：System 1 读中文约快一倍，便于浏览海量文本

- 讲者的推论是：如果我们是中国人、是中文 native speaker，他会选择用中文写文档。
- 理由：他的 **System 1** 读中文比读英文快。他读英文已经比较快了，但读中文仍然比读英文快，大概可能**快一倍**。
- 讲者认为中文这个速度“已经可以了”；但就是这一倍，可以让他快速在文档里面浏览。
- 这对他所谓的 **vision model**（即他的视觉、快速的视觉模型）有帮助：可以快速从海量文字里面……

小结：讲者把 agent native 开发的核心视图落在架构图 + log book/dashboard 上，并明确下一步是 master agent 与学习 DeepSeek Harness。

第 20 部分



好文档要能被快速浏览和定位：视觉快速抓关键信息

讲者认为，“就是这一倍”就能让他在文档里快速浏览；借助他的 **vision model**（视觉快速模型），可以从海量文字里提取关键信息，并帮助定位到想要的位置。

走向世界 ⇒ 中英文双文档必须同时维护，一致性是“地狱和灾难”级问题

- 项目要走向世界，意味着 **DeepSeek Harness**（原声近“deep seek honeys”）要同时维护中文文档和英文文档。
- 讲者反问：这不就是地狱和灾难吗?
- 文档总是随手改、随手提交；当项目里有无穷多文档时，怎么保证中英文文档之间的一致性?
- 讲者说“你们没有想过这个问题，所以这就是经验”。
- 例证是 **docs**（原声“DOS DC”）里有非常多文档。

例子：Architecture.md 的人感与交叉引用合法性

- 看 docs 里随便一个 **Architecture.md**（原声“Architecture 点 markdown1”），是非常有人感的文档，看起来很干净舒适。
- 中间有很多交叉引用。
- 讲者断言：在 **DeepSeek Harness** 里，所有交叉引用都是合法的；找不到任何一个不合法 link。所有文档，甚至文档之间的一致性都是有的。
- 这引出追问：项目里有中文和英文文档，它们如何保持一致，又如何保持人感?

解决方案：三套件 + 一个门禁，把每篇文档变成“同目录三兄弟”的配对契约

逻辑链：中英双文档 → 同目录三兄弟配对契约 → 双侧 hash + git 历史判断新旧 → 不匹配触发 AI patch 翻译 → 程序校验 hash → 一致性。

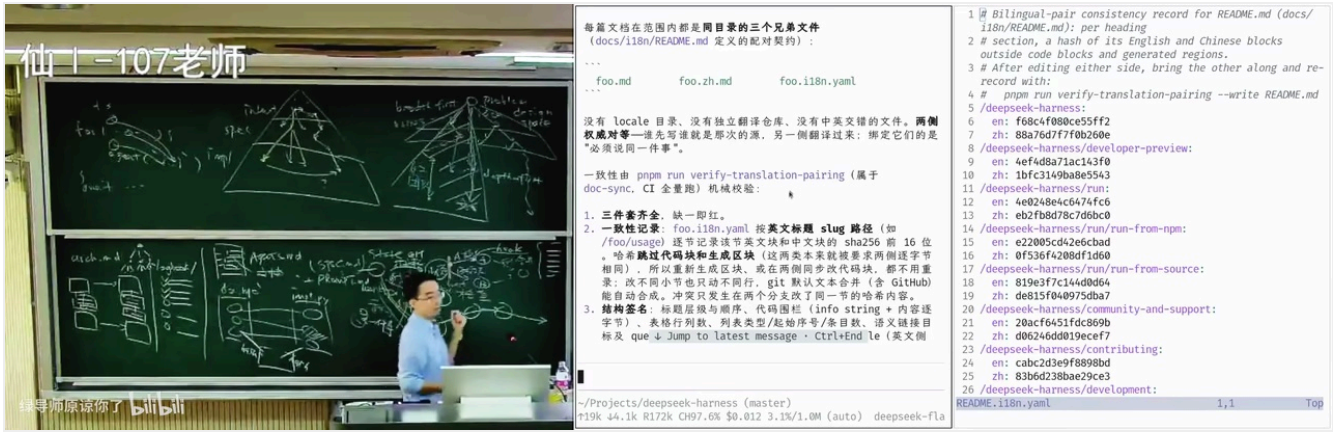
1. 每篇文档在范围内都是同目录的 **三个兄弟文件**，定义配对的契约。
2. 这三个文件包括：中文 markdown、英文 markdown；讲者说中文 markdown 现在还是 **开发者预览阶段**。
3. 还有一个现在不太常见的 practice：**i18n.yaml**（原声“I18N点 yao”，internationalization 的 yaml）。
4. 讲者说“两侧是权威”：谁先写谁就是源，另一侧由 **co-agent** 翻译过来。
5. 绑定它们的要求是：必须做同一件事。

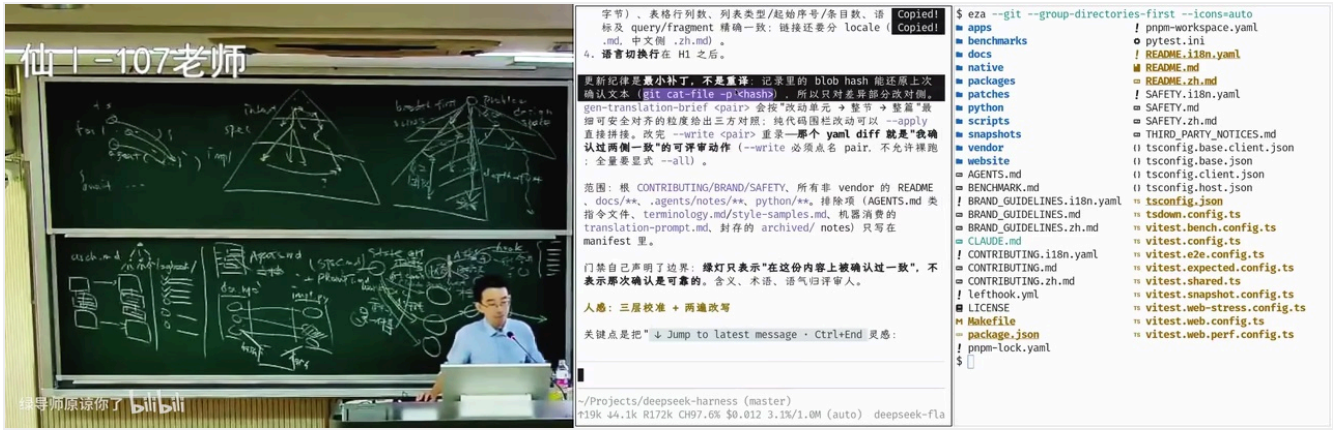
6. 一致性由 `pnpm run verify-translation-pair`（原声近“p m p m run verify translation Parry”）检查。
7. 办法：在另外的文件里放一个 **hash**，两边各放一个 hash。所有过往都留在 **git 版本管理历史**里，所以知道谁变了、谁先变、谁没变、谁更新谁更旧。
8. 类似 `git reflog`（原声“get it ripple”）的新旧判断：旧的 hash 是见过的，新的 hash 是没见过的；在工作区或更新侧，能判断谁新谁旧。
9. 一旦发现新旧版本不匹配，就用 **AI 翻译**。
10. 翻译也是 **patch-based**：看这个文档改了什么，把另一份文档对应位置也改掉，其他地方保持一致。
11. 有些东西由程序保证，比如 hash。例如 English/中文两侧对应的 hash 会放在文档中（原声“DEEPC和list ENGLISH的这个中文的hash在这里”）。这段很短的文本就能保证一致性。

（整理者串联） 以上机制形成一条链：配对契约 → hash/git 新旧判断 → AI patch 翻译 → 程序校验，用来保证中英文文档一致性。

小结：讲者用三套件加一个门禁，把中英文文档一致性交给配对契约、hash/git 判断和 AI patch 翻译来保证。

第 21 部分





Hash 让短文本也能保持一致性

- 讲者先举 **hash**：即使是一段很短的文本（英文/中文对应内容），也可以保证其一致性。
- 这为后面 **i18n** 文档的校验、差异更新做铺垫：内容短不等于不可校验。

软件项目不该被表示成文件系统里的 repo，而该由关系组成

- 讲者认为：软件项目里所有东西都是错综复杂的**关系**，不适合、也不应该用一个文件系统里的 **repository** 来表示。
- **DeepSeek Harness**（原文"deep sick harness"）做了一个牺牲：牺牲目录里的**干净**。
- 代价是文件系统层面看起来乱，但可以通过 IDE 的**投影/过滤**补回体验：
  1. 用 VS Code 等 IDE 配置，忽略不想看到的文件。
  2. 例如忽略所有 `.i18n.*` 一类文件（原文"点 I18N 点样貌"），只留 **documentation**。
  3. 中国开发者可隐藏英文文档，只留中文；英语母语者可忽略中文，只留英文。
  4. 结果：开发者在 IDE 里得到熟悉、一致的开发体验，但底层仍是 **agent-native** 的东西。
- 讲者结论：软件不应该是一个 **repo**，而应该由**关系**组成。

更新只做最小差异补丁，不重新翻译

- 讲者指出：它的更新粒度最小。**补丁不是重新翻译**；只对差异部分相对地改。
- （整理者串联）这与 hash/关系化存储相配合：差异可定位，旧内容不必全量重做。

"人感"作为可校准规范：AI 写的文档也要能被人维护

- 讲者说它把"**人感**"当成可校准的规范。

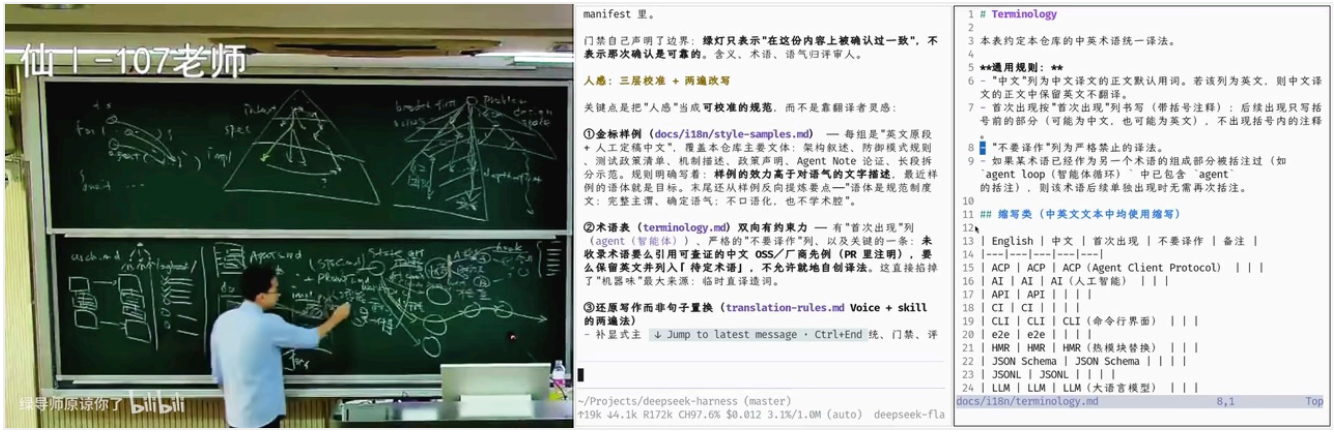
- 在 **i18n** 下有很多文档，例如 `style samples.md`（原文“style3samples 点 MD”），仍然有一些 **AI 味**。
- 这些文档“毫无疑问不是人类写的”，但在项目大到这个体量时，依然有人敢说人能维护；讲者认为这里面下了很大功夫。
- 有些案例是**按需加载**的。
- **渐进式披露**：如果把所有 specification（比如“文档该怎么写”）都塞进 `AGENTS.md`（原文“AGEN 点 MD”），随着内容暴涨，甚至 **100 万上下文**都装不下。
- 所以这些规范被放在 i18n 下，只有需要翻译文档或写文档时才被加载/暴露。

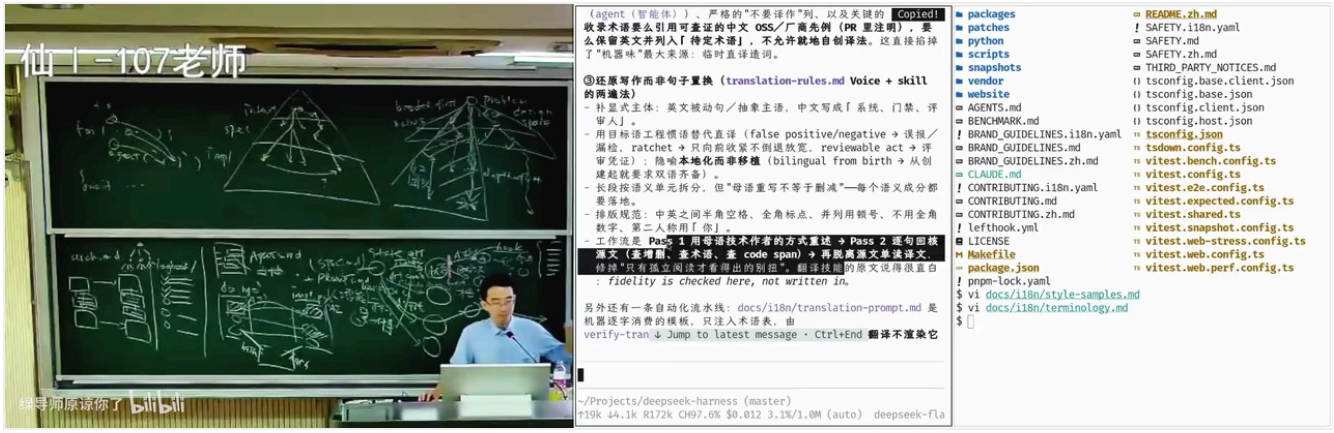
Terminology：统一译法可以用门控做部分检查

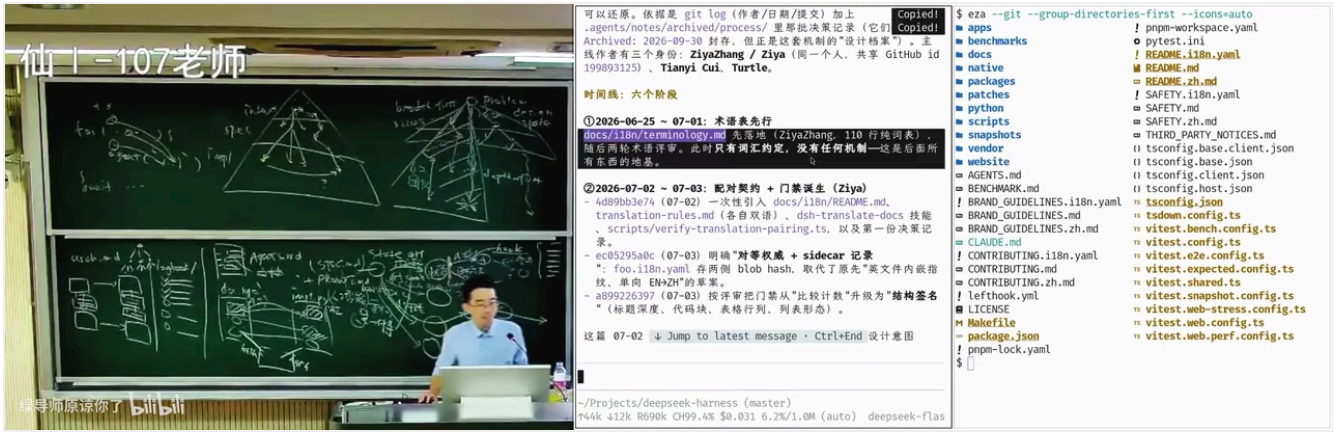
- **Terminology**：约定本仓库中中英文术语的统一译法。
- 讲者说这可以用**门控**在一定程度上检查。例子：
  - 要求中文和英文翻译在 **paragraph** 或 **section** 级别对应。
  - 对特定词，如 **ACP** 或 **AI / 人工智能**，要求出现频率不要超过某个百分比，或数量差在 **±3** 左右。
  - 若 **AI** 对应中文“**人工智能**”，则 AI 与“人工智能”的出现次数需大致匹配（例如 ±2–3）。
- 但讲者提醒：要小心调整**决策边界**，不要把实际可以通过的筛掉。

小结（整理者串联）：讲者主张用关系组织软件，再用 IDE 投影、差异补丁、渐进披露和术语门控，把关系重新变成可维护、可校验、可用的开发体验。

第 22 部分







不约束 agent 就会光速生产 Slop，所以门控边界必须调准

讲者的机制判断：

1. Agent 不受约束时，**文档漂移**太常见。
2. 漂移的后果是：它会“以光速生产 **Slop**”。
3. 因此要设置**门控/门禁（gates）**；但门控的**决策边界**要小心调，不能把实际上可以通过的也筛掉。
4. 门控调好后，就能带来非常好的**文档一致性**。

例子：看 **DeepSeek Harness**（音频中作 depi harness / deep pick harness / deepi honeys 等），里面有“非常非常多的门控”，所以“非常非常的稳”。

DeepSeek Harness 的开发史可还原整套文档一致性流程

讲者提出要回答：这套文档一致性流程由谁、在什么时间引入，又经历了怎样的几次大变化。讲者说 DeepSeek Harness 给了“一个礼物”：所有开发历史应该都在。

- 起点：**2026年6月10号晚上11点**，项目从零开始；讲者称可看到“从空到成长的过程”，到今天已有一定成熟度。
- 主要人物：**Turtle**、**张子雅**、**崔天翼**；讲者说这三人都在他们的内测群里。

2026-06-25：terminology 先行，但还没有机制

- 到 **2026年6月25日**，做法是 **terminology 先行**：没有任何机制，只有**词汇的约定**。
- 讲者判断：这说明他们意识到文档的重要性；也“一定说明他们是有经验的”。

7月：配对契约 + 门禁诞生，并被强行塞进自动化流水线

- **7月份开始**，他们引入**配对契约**（原音“配对的企业”；后文明确为“配对契约加门禁”）。讲者认为这是一个“非常酷的设计”。
- 讲者不确定他们从哪里看到类似东西，猜是否从某个项目看到过。
- **7月3号**：出现“**对等权威塞卡**”（原音，含义待核）。
- 随后讲者看到：这个机制要**强行进入自动化流水线**；文档在很短的早期开发阶段就已形成，并一直遵循。
- 结果之一是：可以同时维护一份**中文和英文**的文档。
- 讲者认为里面有很多“宝藏”。他先想说这是按 CI，随即改口，应说这是 **agent native 开发**。

配对契约不是横空出世：pre-LLM 时代已有代码/文档同步机制

讲者用 **pre-LLM 时代**（原音近 PREMM）的机制类比：

- Python 文档由 **docstring** 生成，例如 **pydoc**；文档和代码可以永远保持一致。
- 这不是 Python 第一个做；每种语言都有类似 **Javadoc** 的语法。
- C++ 原生不支持，但有 **Doxygen**：自己规定注释语法，按规范写注释就能被 doc 工具提取。
- 这样代码和注释、代码和文档，或者说 **API reference**，可以保持 **100% 一致性**。

讲者说，受到这些启发，其实应该早有类似东西；但在 **agent / agent native 时代**，要把人的工作迁移给 agent，**配对契约 + 门禁**就是非常酷的机制。（整理者串联）这也是为什么他把

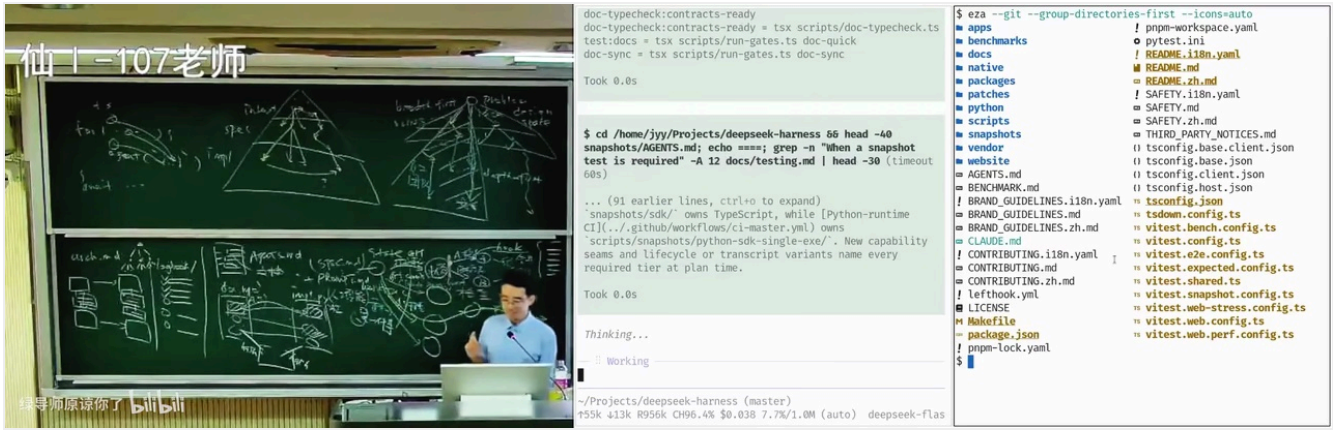
DeepSeek Harness 看作“第一眼就和别人不一样”的项目：它把旧时代的代码/文档同步思想，改造成了 agent 可执行、可门控的流水线。

讲者评价：DeepSeek Harness 是中国团队开源经验的骄傲

- 讲者认为 **DeepSeek Harness** 是“我们中国人的骄傲”；**DeepSeek** 也是骄傲。
- 对比之下，讲者说 **OpenAI** “根本就是 close AI”，只放了一些最基础的东西。
- 讲者强调：DeepSeek 把这样的东西开源了，这件事值得强调。

小结：讲者的主线是，文档一致性不能只靠人的自觉，而要用可自动化、可门控的配对契约来防 agent 漂移；DeepSeek Harness 的开发史正好给出了从 terminology 到配对契约+门禁的演化样本。

第 23 部分



开源与封闭的对比：公开的只是基础门禁，内部 agent review 不露面

- 讲者认为 **DeepSeek** 是“我们中国人的骄傲”，因为它把东西开源了；同时用玩笑口吻说 **OpenAI 就是 close AI**。
- 讲者批评/对比说：公开出来的只是最基础的东西，比如**lint 跑过、CI 里 test 跑过**，即最基本的门禁。

- 但讲者判断，它们在内部服务器上跑了不知道多少轮；并且一定有隔一段时间的 **agent review**。
- 讲者引用对方博客：他们一开始尝试**每周做一次 agent review**，来**防止架构崩溃**。
- 讲者说这些过程“全都消失了”，只把最后能拿出来用的**编译产物**拿出来。
- **逻辑链（讲者陈述）**：公开 repo 只给基础门禁 → 内部 agent review / 防架构崩机制不公开 → 外界看到的是最终可用产物，而不是维持它的过程。

Agent 原生仓库的常驻指令体系：AGENTS.md 分层、文档预算、agent notes

- 对方有一套 **agent native 常驻指令体系**。
- **AGENTS.md 分层**：每一个层级/目录上都有 AGENTS.md（转录原词近似 HMD，按上下文整理）。
- **verify doc budgets**：要求 `root ≤ 1960 次`、`architecture ≤ 2400 次`（“次”为转录原词；讲者上下文是文档预算）。这是**防止 AI 输出/文档无限堆积**的机制。
- **agent notes**：里面有 **implemented / proposed / rejected / archive**。讲者称之为“他们的 rap loop”（[rap loop] 保留原词；疑为 Ralph loop，但转录不确定）。
- 讲者比较自己的做法：我们的 loop 稍微不一样，是自己实现的；有一个 **logbook**，里面 log 会带一个 **status**，并且会和 **GitHub CI** 整合在一起。
- 对方也类似；讲者推测对方可能也可以有 **dashboard**，但他认为对方没有 dashboard，可能觉得不是很有必要。
- 当 agent 需要开发的时候，他们会把它做在一个类似的 logbook 里。
- 对方还有很多 **skills**。

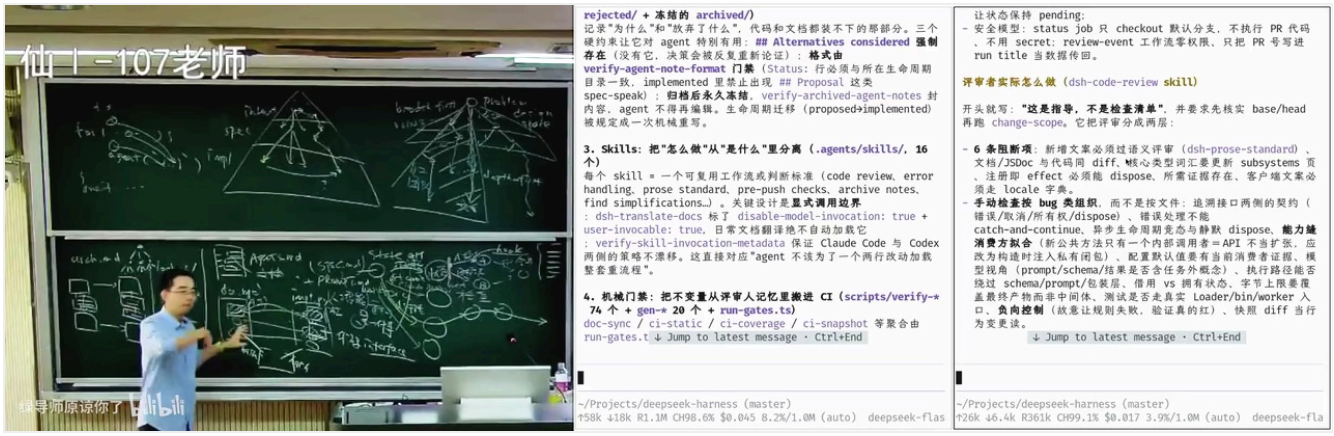
ChatGPT 输出形态变化：动态 UI 组件、HTML 小卡片、Web UI 可能成主流

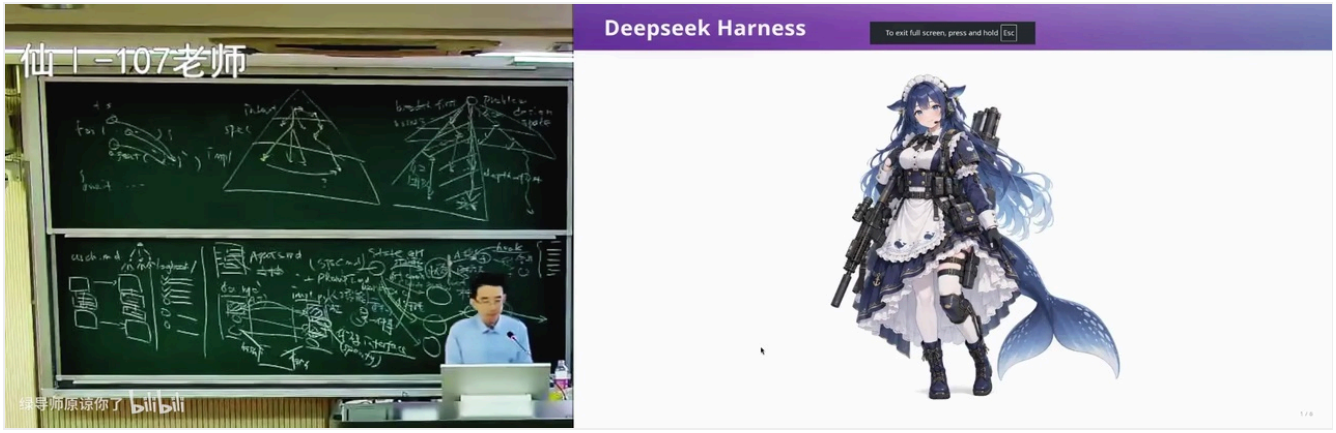
- 讲者观察今天还有一个变化：**ChatGPT** 现在开始喜欢生成一些**动态 UI 组件**，比如一些 **HTML 小卡片**。
- 以前它会把图片堆在前面；现在会把图片**嵌入到文本的流程里**，读起来舒适很多。
- 讲者说，有时候在觉得合适的时候也会这样；未来很快就会来。**移动端**（转录原词：mini 行）可能稍微麻烦一点。
- 讲者判断：**Web UI 很有可能成为主流**。

Code review 如何做：元级认知 + 多层门禁 + 文档/代码一致性

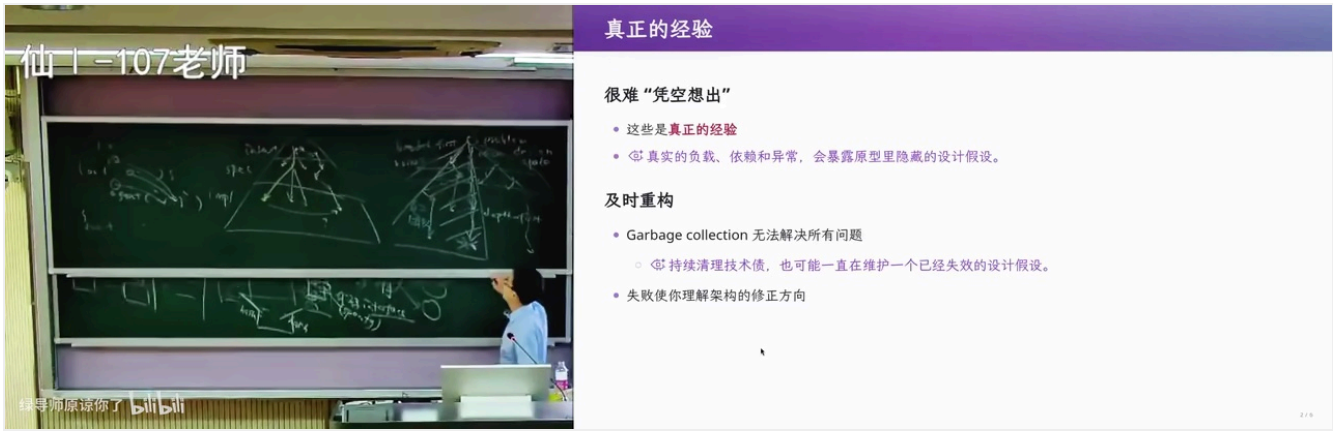
- 讲者提问：**code review 是如何做的**?
- 以前面对一个陌生项目，不可能像现在一样；现在你只要有一些**元级认知**——知道什么东西是好的、什么东西是自己没见过的——就可以**光速向别人学习**这些真正好的东西。
- 评审流程的门禁包括：**本地自证、CI 门禁、人工评审**，而且还有**分数**。
- 还有各种规范与产品自己提供的 **skill**：**code review 写作规范、工程质量规范**等。
- 新增文案必须通过**语义评审**；文档方面还有 [jazz stock]（转录不清，保留原词）。
- **文档一致性检查**：
  - 中文英文要保持一致；
  - 所有链接要一致；
  - 如果文章里有一个代码块，比如一个 **interface**，还会检查这个 interface **原封不动、每一个字符逐字**出现在某一个文件里。
- 目的：防止代码改了那个接口但文档没有 document，或者文档改了接口设计但代码没有跟上。讲者说要“**绝对杜绝这种情况**”。
- 讲者最后说：其实我们把这个想法稍微做了一点扩展。

第 24 部分









文档没有 document 或接口设计变了，代码必须同步

- 讲者先立了一条硬规则：如果**文档没有 document**，或者**文档改了接口设计**，而**代码没有跟上**，就“绝对杜绝这种情况”。
- **（整理者串联）** 这实际上是在防止 **文档—代码漂移**：接口设计一变，代码必须同步，不能文档说一套、代码做一套。
- **（整理者串联）** 这也解释了后文为什么要用大量 CI/检查把一致性机械化：靠人记不可靠，必须让漂移被门禁拦住。

code block 带执行语义：把 spec 和轻量检查写进 Markdown

- 讲者说，他们把这个想法在 **agent native 项目**里稍微扩展了一点：**code block 是带执行语义的**。
- code block 里可以 **specify 一个非常小的说明**，表明“我这段东西是干嘛用的”。例如：
  - 我需要它出现在某一个位置；
  - 我希望它以某一种形式和另外一个东西关联起来。
- 里面还有一些**轻量级 check**，扩展了 **Markdown fenced code block / code block** 的语法（原音频近似“markdown 的 fans code block / gold block”）。
- **Code review** 也是一个环节；之后继续用 **skills** 分离出各种机械门禁：**Doc sync CI**、**static CI**、**coverage CI**、**snapshot** 等。

机械门禁的代价与收益：提交前多跑检查，换来项目被维护住

讲者给出的逻辑链可以整理为：

1. code block 带执行语义，可写小 spec 和轻量 check。
2. 这些 check 进一步被拆成机械门禁，如 **Doc sync CI**、**static CI**、**coverage CI**、**snapshot** 等。
3. 代价是：每一次提交之前，可能都要运行大量门禁。
4. 但在 agent native 工作方式下，“你又不写代码，大部分时候你都是监工”；你写 **plan**，和 AI 一起写 plan。
5. 具体做法：在 **log book** 里写 plan；在 **plan mode** 先搞清楚要做什么、做到什么程度；剩下的事情不是人做的，是 AI 做的。
6. 所以改 log book 时不需要运行那些 CI：**所有代码都没有动过**，因此不需要运行任何 CR/检查。
7. 单个门禁看起来都非常小，但把它们全部加起来时，就会发现：**项目被维护起来了**。

需要运行的验证包括：

- **verify client**、**UI**、**WiFi translation**、**pairing application entry points**；
- 一些 **TypeScript script** 相关检查；
- 其中一个关键是 **verify type equivalence**（原词近似“verify type equivalent”）：**文档里粘贴的类型必须与源码一致，且不允许漂移**。

讲者还提到还有很多类似机制，比如**文档分层**、**从任务到机制的路由**（原音频近似“税任务到机制的路由”）等。讲者把这一整套称为 **agent native**。

这些 agent native 经验来自真实实践，不是教室里推演出来的

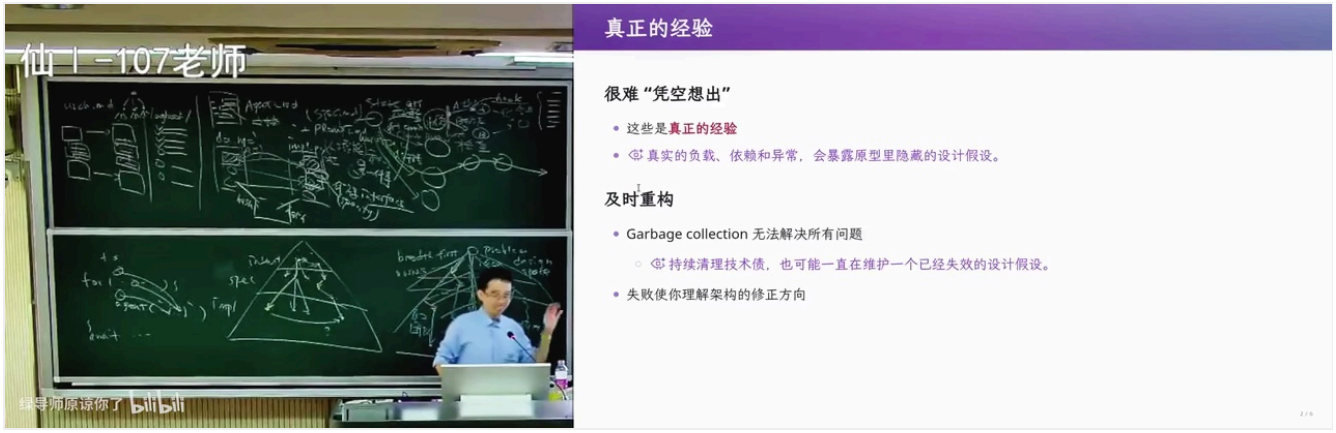
- 讲者从 agent 时代的时间线说起：从去年到今年，尤其到今年年初，大家发现 agent；短短半年。
- 这个项目从 **6 月份**开始，就已经在规划这些事了。
- 讲者认为他们看到了一个非常好、非常有启发性的东西，并且它是**开源**的。
- 这些做法来自**真正的经验**：你一看就知道“确实应该是这样”，但很难完全凭空想出。
- 如果只是在教室里做**语言律师**，思维会被当前上下文禁锢，只能不停往下走；你不会发现 **design space / 搜索空间**里还有巨大空间，可以朝另一个方向探索。
- 讲者认为这些人肯定失败过、写过代码；他们的年龄可能和讲者差不多，是同龄人。因此他们有这些经验。

架构烂掉后的重构策略：新开空白项目，逐步迁移旧功能

- 讲者说，即便如此，大家做项目时可能都逃不过重构：最终有一天你会发现项目烂掉了。
- 他讲了自己对学生说的话：因为那个项目的 **architecture** 烂掉了，所以：
    1. 先开一份新的、白的[项目]；
    2. 然后一点一点把旧的功能迁进来；
    3. 保持一个好的架构。

小结：讲者主张用带执行语义的 code block 和拆碎的机械门禁，把 agent native 项目的维护变成可自动检查的约束；经验来自实践，架构烂掉时宁开新白项目逐步迁移。

第 25 部分



方向不对时，重构比垃圾回收更根本：开新空白项目，逐步迁移旧功能

- 讲者的建议：先开一份新的空白（项目），然后把旧功能一点一点迁进来，保持一个好的架构；他用“be like 好”表示认可这种说法。
- 在 **DAFIRST（原词，疑似 design first）** 阶段，如果发现矛头不对，**garbage collection（垃圾回收）** 可能无法解决所有问题。
- 所以必要时要做**重构**：可以**模块级重构**，也可以**整个项目级重构**。讲者认为这能让项目走上更正确的方向。
- （整理者串联）这里的关键不只是清理垃圾代码，而是纠正项目的架构与方向。

历史失败（Friendster）被讲者当作 agent-native 开发经验的来源

- 讲者举社交网络先驱 **FRANSTER（Friendster）**：它迅速吸引大量用户，然后价格崩溃。
- 讲者说这是 **PREMM（原词）时代** 的事；在今天 **AI agent（原词 A 证）** 时代，可能 AI agent 能把它富起来。
- 讲者认为，这些经验最终促成了一个 **agent-native 开发** 的好范例。

威胁一：不反复试错、失败、建东西，好 intent 也会变成 AI slop

- 讲者判断：你不做很多次、不失败很多次、不建很多东西，就不太可能把一个东西真正做好。

- 你可能确实有一个好想法，在 **intent** 方面是世界级的。
- 但如果组一个**草台班子**去做，讲者说可能进入“一个直线”，海量产生 **AI slop**，最后好想法被别人抄走。
- 反之，如果有**有经验的团队**，更可能**收敛到一个更好的方向**。
- 讲者的链：好 intent + 草台班子/缺少试错 → AI slop → 被抄走；好 intent + 有经验团队 → 收敛到更好方向。

威胁二（社会性）：没有 token 就没有足够试错机会

- 讲者提到开学时随口说“没有 token 的学生应该立即退学”，并说明这是**玩笑**。
- 但真实发生的事是：**没有 token，尝试的机会就不如别人多**。
- 不去经历试错过程，就只能看别人；最后看你的**悟性**，能否通过看别人 **open source（原词 open 手）** 的东西学会。现在 open source 还有好的东西。
- 讲者认为，即便如此，你还是需要做很多尝试、失败。
- （整理者串联）token 在这里不只是工具成本，而是试错机会的分配条件；token 不平等会转化为能力积累的不平等。

斩杀线：读完博士不如 AI agent 好用，用人单位会选 senior + 无限 token

- 讲者问：如果读完博士时依然不如 **AI agent** 好用，用人单位会做什么？
- 讲者判断：他们会选一个 **senior**，给他**无穷的 token**，而不是培养一个 **PhD 学生**。
- 理由：如果是资本家，要最大化社会/企业的运行效率。
- 以前可以“师傅带徒弟”，招新兵蛋子，看潜力。
- 但如果你的**上升/回报点太长**，比如需要 **4–5 年**，公司早就黄了，你才达到 agent 的水平。
- 同时 agent 现在就在这里，而且 AI agent 实际上也在上涨；所有人都在被不断“斩杀”。
- 讲者的链：培养 PhD 需 4–5 年 → 回报周期长 → 公司/资本等不起 → 直接选 senior + 无限 token。

斩杀线继续下移：本科生毕业还有没有 $20 Claude Pro 的价值?

- 讲者说这条**斩杀线**现在越过，比如**本科生毕业**的时候：你还有没有一个 **$20 的 Claude Pro** 的价值?
- （整理者串联）这等于把同一套性价比逻辑下移到本科毕业：若你的作用不如每月 $20 的 Claude Pro，雇佣/培养你的经济理由就更弱。

第 26 部分



20 美元 Claude Pro 订阅：Opus 4.5 的额度够用且能力很强

- 讲者说自己现在有一个 **20 美元 Claude Pro 订阅**，在 Anthropic 账号和某个工具（原文“ground bot”）里养自己的 **Claude Code**。
- 他发现 **Opus 4.5**（原文“OPPOS5.5”）的额度其实还挺够用，而且 Opus 4.5 真的很强。
- 他由此发问：本科生毕业的时候，是否还值一个 **20 美元 Claude Pro** 的钱？他的判断是：本科生要达到 Opus 4.5 的水平，“很困难，非常困难”。

有 big picture 的 senior + 好的 agent harness，是另一条优势条件

- 讲者设想：如果自己是 **senior**，有比较大的 **big picture**，又有一个好的自己的 **agent harness**，情况就不同。
- （整理者串联）这里与上一条形成对照：单纯“本科生水平”容易被模型追上，而全局判断加工具编排可能仍是讲者眼中的优势。

闭源大厂把能力锁在内部，会造成“越来越多人被斩杀”

- 讲者指出：像 OpenAI、Anthropic 这种东西从来没有开源出来。
- 还要靠“国产之光” DeepSeek（原文“d pk”）来开源。
- 他的推理链：
  1. 如果这些能力都留在公司内部；
  2. 并且永远不让别人知道；
  3. 那就会**越来越多的人被斩杀**。
- 讲者把这称为一个**社会性问题**。

理想解法可能在大学，但现实大学也是资本化、KPI 化、学阀化

- 讲者认为，这个问题的解决**可能只有在大学里能解决**，因为大学是**非盈利**的，不是以盈利性质或 KPI 为导向的地方。
- 但他马上转折：“话是这么说”，现实里大家都清楚：**学阀一手遮天**，网上能刷到这类故事。
- 他用自己的处境说明：考核指标不是上课，而是所谓的**科研经费**。
- 所以高校某种程度上也是**资本家和公司化运营**，越来越**资本化**。
- 区别在于：这是另一套资本体系，不是外面的资本体系，而是由 **基金委、科技部、优青、杰青、院士** 等构成的**金字塔**。
- 当这个金字塔下面每一个 **杰青**（原文“结晶”）的产出都类似公司产品时，体系就按公司逻辑运转。

学生追不上 agent → 老师学生被斩 → 旧体系失效 → 大学崩溃

- 讲者给出的链条：
  1. 当**学生永远赶不上 agent**；
  2. 老师和学生都被“斩”；
  3. 这个体系就完全失效。
- 因此他的结论是：**大学肯定要崩溃**。他自称这又是在“发表暴论”。

学而思被讲者看成“新形态大学”：样板课 + 资本化运营 + AI 执行

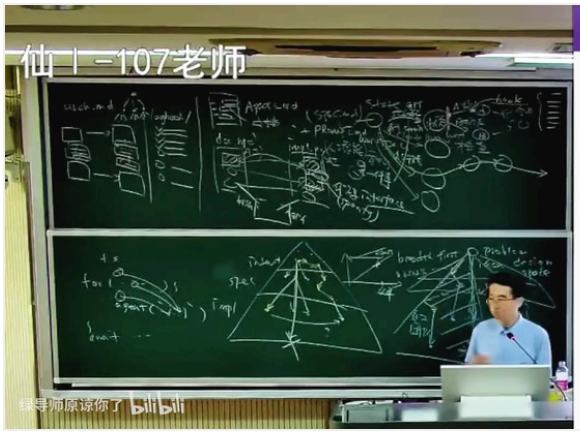
- 讲者反而觉得，像**学而思**这样的东西可能是一种**新形态的大学**。
- 理由是它从上到下都是**资本化运营**。
- 它靠的是**样板课**，而且样板课很牛逼。

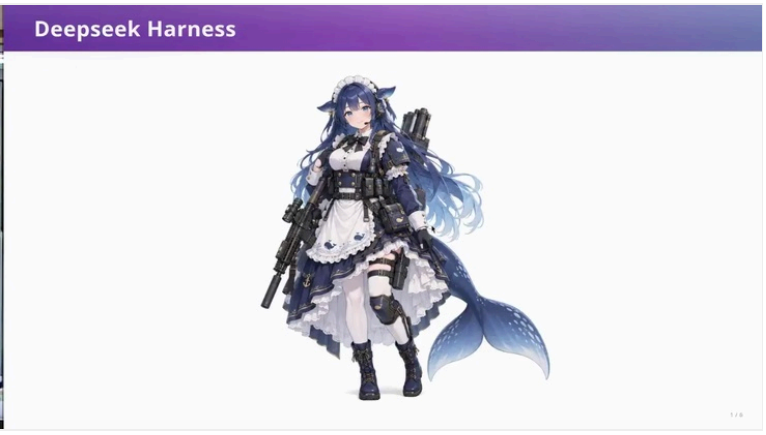
- 具体做法：
  - 花 100 万雇一个清北天才来设计样板课；
  - 上样板课、试；
  - 试完以后，**每一个老师都是执行的机器**。
- 讲者进一步说：现在把老师的执行机器换成**老师 + AI**。
- 它还花 1000 万去“技摩场”（原文如此）抢最好的天才，然后设计样板课。
- 结果是：它把**教学、教育体系完全颠覆**。
- 对比传统大学：我们没办法，因为**惯性很大**，还得这样上课，还得用这个方式上课；而学而思因为是资本化运营，所以能这样做。
- 对“未来会发生什么”，讲者说：**我不知道**。

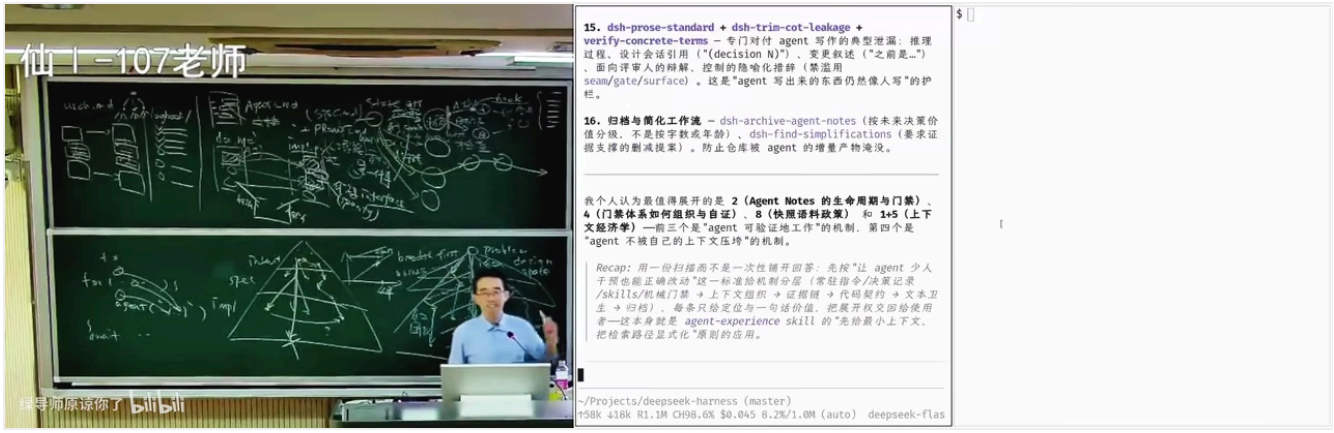
开源社区给了讲者今天的理解；建议去互联网找真正优秀的项目

- 讲者非常感谢**开源社区**。
- 他说：尤其是我自己，我自己的这一切都是开源社区给的。
- 如果不在开源社区里看到那些好的东西，他就不会有今天的理解，也不会来上这个课。
- 因此他的建议是：大家现在可以回去，在互联网上找一找那些**真正优秀的项目**。

第 27 部分







推荐入口：找真正优秀的项目，从 DPC Harness 看起

- 讲者认为，现在大家已经可以回去在互联网上找那些真正优秀的项目。
- 他给出的具体起点是 **DPC Harness**（原文作 DP c harness / DPCHNEX / honeys）。
- 理由（讲者陈述）：这个 **Harness** 和他今天课上演示的东西本质上是一类东西——都围绕一个很小的 **RL loop** 来组织。

今天的 demo：小到不能算标准 RL loop，重点在“一切皆插件”

- 讲者对自己今天演示的定位：它是一个“特别特别小”的 **RL loop**。
- 但严格说，它不能算 RL loop，更像一个 **A正点P**（原文如此；术语不确定，保留原词）。
- 他又说，可以把它理解成 **DPC Harness**。
- 它的关键特征是：**一切皆插件**，所有东西都插件化了。
- 讲者认为这里面有“一些有趣的东西”。

逻辑链

1. 真正优秀的项目值得去互联网上找。
2. 找的时候可以从 **DPC Harness** 开始看。
3. 因为 DPC Harness 与今天那个极小 demo 本质同类：都是 RL loop / 插件化结构。
4. 今天 demo 太小，甚至不能算标准 RL loop，只能算 **A正点P**；但可以把它当作 DPC Harness 的简化理解。

5. **（整理者串联）** 先看成熟项目，再回看今天的小 demo，更容易看清“一切皆插件”的设计如何支撑 RL 循环。

小结：讲者最后把 **DPC Harness** 作为成熟参照，并强调今天的小 RL 循环核心是插件化。
